



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير إنجاز مشروع مادة البرمجة المتوازية في الهندسة المعلوماتية

High-Performance E-Commerce Backend Engine

العام الدراسي

2026 - 2025

الملخص

شهدت أنظمة التجارة الإلكترونية خلال السنوات الأخيرة نمواً كبيراً في أعداد المستخدمين وحجم العمليات المنفذة في الوقت نفسه، مما أدى إلى ظهور تحديات تقنية تتعلق بالأداء، وسلامة البيانات، وإدارة الموارد الحاسوبية. وفي الأنظمة ذات الأحمال المرتفعة لا يكفي توفير الوظائف الأساسية فقط، بل يصبح من الضروري ضمان قدرة النظام على التعامل مع أعداد كبيرة من المستخدمين المتزامنين دون التأثير على استقرار البيانات أو سرعة الاستجابة.

يهدف هذا المشروع إلى تصميم وتنفيذ محرك معالجة عالي الأداء لمنظومة تجارة إلكترونية مع التركيز على المتطلبات غير الوظيفية المرتبطة بالأنظمة المتوازية والموزعة. تم تطوير النظام باستخدام إطار العمل Laravel Octane، مع الاعتماد على Redis لتخزين البيانات المؤقتة وتسريع الوصول إليها، بالإضافة إلى قاعدة البيانات MySQL لتخزين البيانات الدائمة.

يتضمن المشروع تطبيق مجموعة من المفاهيم المستخدمة في الأنظمة الحديثة عالية الأداء، مثل التحكم بالتزامن بين العمليات، ومعالجة المهام الخلفية، ومعالجة البيانات على دفعات، وتوزيع الأحمال على عدة خوادم، والتخزين المؤقت للبيانات، وضمان سلامة المعاملات داخل قاعدة البيانات.

تم إجراء مجموعة من اختبارات الضغط باستخدام أداة K6 لمحاكاة أكثر من مائة مستخدم متزامن، بالإضافة إلى تنفيذ سيناريوهات عملية لإثبات وجود مشكلة التضارب بين العمليات المتزامنة والتحقق من نجاح الحلول المطبقة لمنعها. كما تم تنفيذ اختبارات مقارنة لقياس أثر معالجة البيانات على دفعات في تحسين الأداء، ودراسة تأثير التخزين المؤقت في تقليل الحمل على قاعدة البيانات وتسريع الاستجابة.

أظهرت النتائج النهائية قدرة النظام على التعامل مع الأحمال المرتفعة مع الحفاظ على سلامة البيانات ومنع التضارب بين العمليات المتزامنة، إضافة إلى تحقيق تحسن ملحوظ في الأداء وسرعة الاستجابة من خلال استخدام التخزين المؤقت، وتوزيع الأحمال، ومعالجة المهام الخلفية. وتؤكد هذه النتائج نجاح النظام في تحقيق المتطلبات غير الوظيفية المطلوبة ضمن بيئة تحاكي ظروف التشغيل الفعلية لأنظمة التجارة الإلكترونية الحديثة.

المقدمة

أصبحت التجارة الإلكترونية أحد أكثر القطاعات اعتماداً على الأنظمة البرمجية الحديثة، حيث تعتمد آلاف الشركات حول العالم على تطبيقات الويب والمنصات الرقمية لإدارة عمليات البيع والشراء وخدمة العملاء. ومع ازدياد أعداد المستخدمين وارتفاع حجم الطلبات اليومية، برزت الحاجة إلى تطوير أنظمة قادرة على العمل بكفاءة عالية تحت ظروف التشغيل المكثفة والأحمال المرتفعة.

تواجه أنظمة التجارة الإلكترونية تحديات عديدة لا تتعلق فقط بتوفير الوظائف الأساسية مثل إدارة المنتجات والطلبات والمستخدمين، وإنما تتعلق أيضاً بالمتطلبات غير الوظيفية التي تؤثر بشكل مباشر على جودة النظام واستقراره. فعلى سبيل المثال، قد يحاول عدد كبير من المستخدمين شراء المنتج نفسه في اللحظة ذاتها، الأمر الذي قد يؤدي إلى تضارب في البيانات وحدوث أخطاء في المخزون إذا لم يتم تطبيق آليات مناسبة للتحكم بالتزامن.

بالإضافة إلى ذلك، تتطلب الأنظمة الحديثة القدرة على تنفيذ عدد كبير من العمليات في وقت واحد مع المحافظة على سرعة الاستجابة وعدم استهلاك الموارد الحاسوبية بشكل مفرط. كما أن بعض العمليات مثل إنشاء الفواتير وإرسال الإشعارات لا تحتاج إلى التنفيذ الفوري ضمن دورة حياة الطلب، مما يستدعي استخدام أساليب المعالجة غير المتزامنة لتحسين الأداء وتقليل زمن استجابة المستخدم.

من ناحية أخرى، يؤدي التعامل مع كميات كبيرة من البيانات إلى ظهور تحديات إضافية تتعلق بكفاءة المعالجة واستهلاك الذاكرة. ولهذا السبب تعتمد الأنظمة الحديثة على تقنيات مثل معالجة البيانات على دفعات والتخزين المؤقت من أجل تحسين الأداء وتقليل الضغط على قواعد البيانات.

يركز هذا المشروع على دراسة وتطبيق مجموعة من المفاهيم المرتبطة بالأنظمة المتوازنة وعالية الأداء ضمن بيئة تجارة إلكترونية متكاملة. وبدلاً من التركيز على عدد كبير من الوظائف أو الواجهات، تم التركيز على تصميم بنية قادرة على التعامل مع التحديات المرتبطة بالتزامن، إدارة الموارد، توزيع الأحمال، والمعالجة الخلفية للمهام.

يتضمن المشروع تطبيق عدد من المفاهيم الأساسية في هندسة البرمجيات والأنظمة الموزعة، بما في ذلك التحكم بالتزامن باستخدام الأقفال المتنافسة والمتشائمة، استخدام الطوابير لمعالجة المهام الخلفية، الاستفادة من Redis كطبقة تخزين مؤقت موزعة، تنفيذ معالجة البيانات الضخمة باستخدام تقنية Chunks، بالإضافة إلى توزيع الطلبات على عدة خوادم باستخدام موازن أحمال يعتمد على استراتيجية Least Connections.

وقد تم إجراء مجموعة شاملة من اختبارات الضغط والأداء للتحقق من قدرة النظام على خدمة أكثر من مائة مستخدم متزامن مع المحافظة على سلامة البيانات وعدم فقدان أي معلومات أثناء التنفيذ.

وبالتالي فإن الهدف الرئيسي من هذا المشروع يتمثل في بناء منصة تجارة إلكترونية عالية الأداء تحقق المتطلبات غير الوظيفية المطلوبة وتبرهن عملياً على فعالية التقنيات المستخدمة في الأنظمة الحديثة ذات الأحمال المرتفعة.

يركز هذا المشروع على تطبيق المتطلبات غير الوظيفية الخاصة بالأنظمة عالية الأداء أكثر من التركيز على زيادة عدد الوظائف أو الصفحات، وذلك بهدف دراسة تأثير تقنيات التزامن، إدارة الموارد، التخزين المؤقت، والمعالجة الخلفية على استقرار النظام وكفاءته تحت ظروف التشغيل الحقيقية.

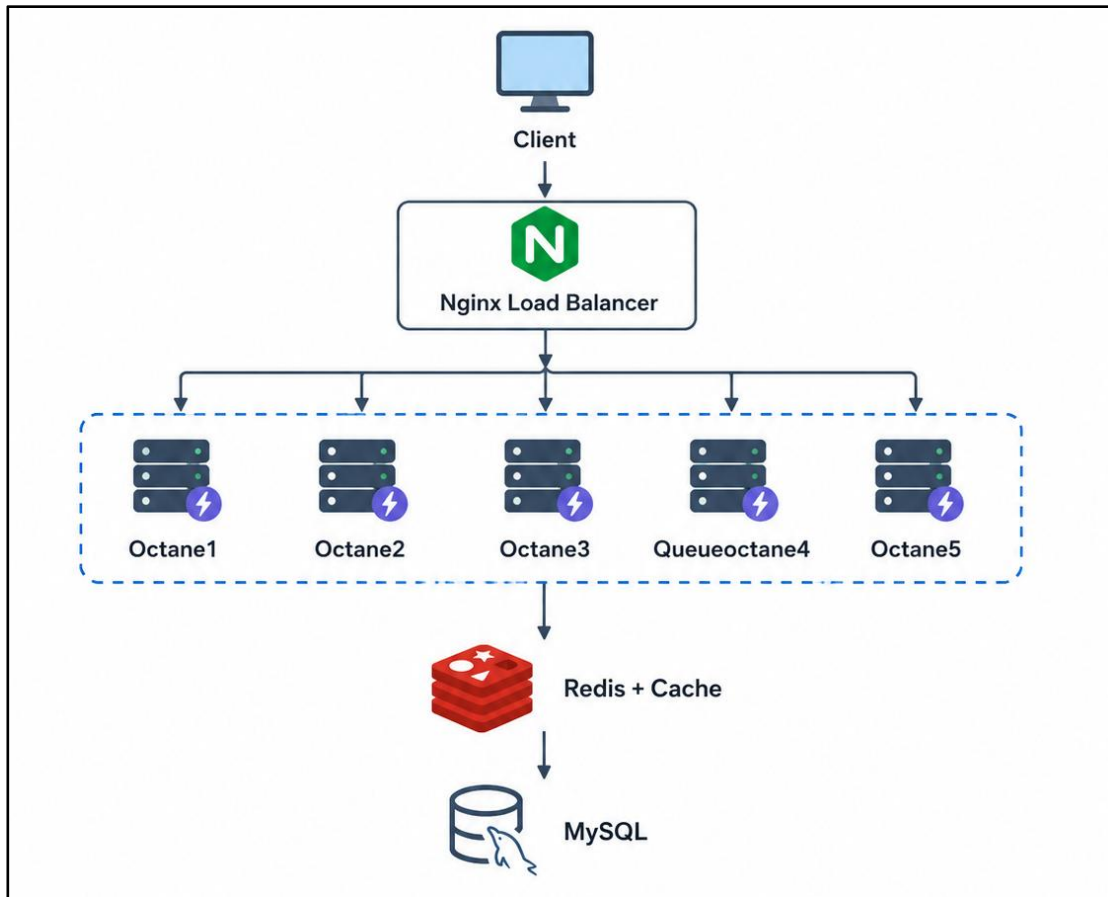
البنية المعمارية للنظام (System Architecture)

تم تصميم النظام وفق بنية متعددة الطبقات تهدف إلى تحقيق قابلية التوسع والاعتمادية العالية وتحسين الأداء عند التعامل مع أعداد كبيرة من الطلبات المتزامنة.

تعتمد البنية العامة للنظام على وجود موازن أحمال (Load Balancer) في الواجهة الأمامية يقوم باستقبال جميع الطلبات الواردة من المستخدمين ثم توزيعها على مجموعة من خوادم التطبيق العاملة باستخدام Laravel Octane. تسمح هذه البنية بتوزيع الحمل على عدة خوادم بدلاً من الاعتماد على خادم واحد، مما يؤدي إلى تحسين الاستجابة وزيادة قدرة النظام على تحمل الأحمال المرتفعة.

بعد وصول الطلب إلى أحد خوادم التطبيق، يتم تنفيذ منطق الأعمال (Business Logic). وإجراء العمليات المطلوبة تعتمد الخوادم على Redis لتخزين البيانات المؤقتة والبيانات عالية الاستخدام مثل سلال التسوق وقوائم المنتجات الأكثر طلباً، الأمر الذي يقلل عدد الاستعلامات المباشرة المرسلة إلى قاعدة البيانات.

أما البيانات الدائمة مثل المستخدمين والمنتجات والطلبات والفواتير فتتم إدارتها باستخدام قاعدة بيانات MySQL. بالإضافة إلى ذلك تم تخصيص خادم مستقل لمعالجة المهام الخلفية والطواير، حيث يتم تنفيذ العمليات غير المتزامنة مثل معالجة الطلبات وإصدار الفواتير وإرسال الإشعارات دون التأثير على سرعة استجابة المستخدم.



الشكل رقم (1)

البنية المعمارية العامة للنظام : (Overall System Architecture)

الشكل رقم (1):

ويتكون الشكل من العناصر التالية:

1. المستخدمون الذين يرسلون الطلبات إلى النظام.
2. موازن الأحمال المسؤول عن توزيع الطلبات.
3. ثلاثة خوادم لمعالجة الطلبات الواردة.
4. خادم مخصص لمعالجة الطوابير والمهام الخلفية.
5. خادم المستخدم للتخزين المؤقت وإدارة البيانات المؤقتة.
6. قاعدة بيانات المسؤولة عن تخزين البيانات الدائمة.

ساهم هذا التصميم في تحقيق الفصل بين مكونات النظام المختلفة، وتحسين قابلية التوسع، وتقليل الضغط على قاعدة البيانات، ورفع القدرة على التعامل مع الأحمال المرتفعة بصورة أكثر كفاءة.

لمحة عن الوظائف الأساسية (Functional Overview)

● تسجيل المستخدمين وتسجيل الدخول باستخدام JWT

يوفر النظام آلية آمنة لتسجيل المستخدمين الجدد وتسجيل الدخول باستخدام (JWT (JSON Web Token، حيث يتم إصدار رمز مميز عند تسجيل الدخول يُستخدم لاحقاً للتحقق من هوية المستخدم أثناء التعامل مع الـ API بدون الحاجة لإعادة تسجيل الدخول في كل طلب.

● إدارة المنتجات: إضافة – تعديل – حذف – عرض

يتيح النظام لوحة تحكم لإدارة المنتجات بشكل كامل، حيث يمكن إضافة منتجات جديدة مع تفاصيلها، تعديل البيانات الحالية، حذف المنتجات غير المتاحة، بالإضافة إلى عرض قائمة المنتجات للمستخدمين بطريقة منظمة وسريعة الاستجابة.

● إدارة سلة التسوق باستخدام Redis

يتم استخدام Redis لتخزين بيانات سلة التسوق بشكل مؤقت وسريع، مما يسمح بإضافة المنتجات وتعديلها وحذفها بكفاءة عالية دون التأثير على أداء قاعدة البيانات، خصوصاً في حالات الاستخدام المتزامن.

● إنشاء الطلبات ومعالجة المخزون

عند تأكيد عملية الشراء، يتم إنشاء طلب جديد في النظام وربطه بالمستخدم، مع تحديث كمية المخزون تلقائياً لضمان عدم بيع منتجات غير متوفرة، مما يحافظ على دقة البيانات.

● إصدار الفواتير وإرسال الإشعارات

يقوم النظام بإنشاء فاتورة لكل طلب مكتمل تحتوي على تفاصيل المنتجات والأسعار، بالإضافة إلى إرسال إشعارات للمستخدم مثل البريد الإلكتروني لتأكيد نجاح العملية.

● إدارة المخزون وتتبع الكميات المتاحة

يوفر النظام آلية دقيقة لتتبع كميات المنتجات في المخزون بشكل لحظي، مع تحديث تلقائي عند كل عملية بيع أو تعديل، مما يساعد على منع النقص أو التكرار في الطلبات.

● عرض المنتجات الأكثر طلباً

يعرض النظام قائمة بالمنتجات الأكثر مبيعاً اعتماداً على عدد الطلبات أو الكميات المباعة، بهدف تحسين تجربة المستخدم ودعم قرارات الشراء من خلال إبراز المنتجات الرائجة.

● دعم المعالجة المتزامنة والطلبات غير المتزامنة

يدعم النظام معالجة عدد كبير من الطلبات في نفس الوقت (Concurrency) باستخدام تقنيات مثل الطوابير (Queues). والعمليات غير المتزامنة، مما يحسن الأداء ويمنع التباطؤ عند الضغط العالي.

المتطلبات الغير وظيفية Non-Functional Requirements

حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

مقدمة

تُعد سلامة البيانات من أهم التحديات في الأنظمة الموزعة وعالية الأداء، خصوصاً في أنظمة التجارة الإلكترونية التي تتعامل مع عدد كبير من العمليات المتزامنة في نفس اللحظة. حيث يمكن لعدة مستخدمين محاولة الوصول إلى نفس المورد (مثل المخزون أو المنتج) في الوقت ذاته، مما يخلق احتمالاً كبيراً لحدوث تضارب في البيانات.

في هذا السياق، تظهر مشكلة تُعرف باسم حالة السباق (Race Condition)، وهي حالة تحدث عندما تعتمد أكثر من عملية على نفس البيانات المشتركة دون وجود آلية تنسيق أو تحكم بالتزامن، مما يؤدي إلى نتائج غير صحيحة مثل بيع كمية أكبر من المخزون الفعلي.

يهدف هذا القسم من المشروع إلى:

- إثبات وجود مشكلة التضارب عملياً تحت ضغط مرتفع
- تحليل سلوك النظام في الحالة غير الآمنة
- تطبيق آلية تحكم بالتزامن تعتمد على الأقفال
- تقييم الفرق في النتائج باستخدام اختبارات ضغط واقعية

وصف المشكلة

لفهم المشكلة عملياً، تم اختيار منتج يحتوي على كمية مخزون أولية مقدارها:

20 وحدة فقط

ثم تم تنفيذ اختبار ضغط باستخدام أداة K6 بمحاكاة:

- 50 مستخدم متزامن
- مدة الاختبار: 20 ثانية
- جميع المستخدمين يحاولون شراء نفس المنتج في نفس اللحظة

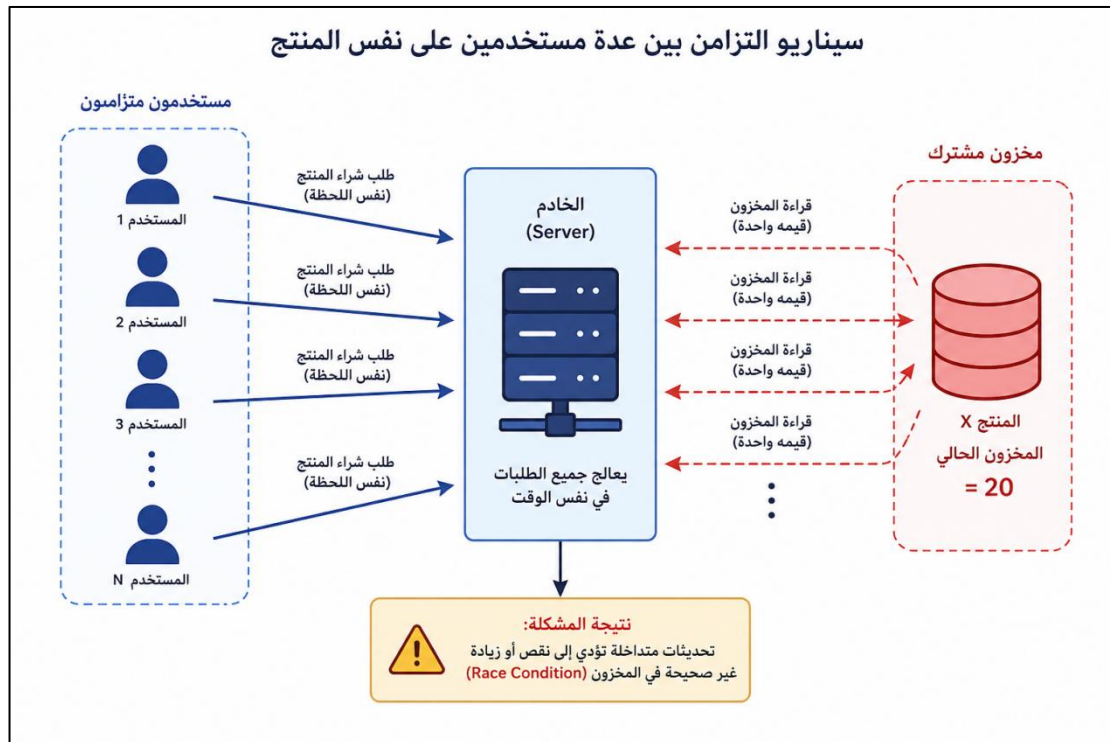
آلية المشكلة

في الحالة غير المحمية، يقوم النظام بتنفيذ العمليات التالية بشكل متزامن:

1. قراءة قيمة المخزون (stock = 20)
2. كل طلب يقرأ نفس القيمة قبل التحديث
3. كل طلب يقلل المخزون بشكل مستقل
4. لا يوجد تنسيق بين العمليات

النتيجة المتوقعة نظرياً:

- بيع أكثر من 20 وحدة
- حدوث تضارب في المخزون
- احتمالية وصول المخزون إلى قيمة سالبة



سيناريو التزامن بين عدة مستخدمين على نفس المنتج

تنفيذ السيناريو غير الآمن (Unsafe Scenario)

تم إنشاء مسار اختبار خاص داخل النظام:

/orders/pessimistic/test/no-limit/unsafe

خصائص هذا المسار:

- لا يستخدم أي أقفال
- يعتمد على قراءة مباشرة للمخزون
- تحديث مباشر بدون حماية
- يسمح بالتنفيذ المتزامن الكامل

من الكود (CartService / Order flow):

```
public function confirmOrderUnsafe(Order $order, array $items): Order
{
    $total = 0;
    $orderItems = [];
    foreach ($items as $item) {
        $product = $this->productRepo->find($item['product_id']);
        if (!$product) {
            throw new \RuntimeException('Product not found', 404);
        }
        if ($product->stock < $item['quantity']) {
            throw new \RuntimeException('Insufficient stock', 409);
        }
        $total += $product->price * $item['quantity'];
        $orderItems[] = [
            'product_id' => $product->id,
            'quantity'   => $item['quantity'],
            'price'      => $product->price,
        ];
        $product->decrement('stock', $item['quantity']);
    }
    $this->orderRepo->update($order, [
        'total_price' => $total,
        'status'      => 'confirmed',
    ]);
    foreach ($orderItems as $oi) {
        $order->items()->create($oi);
    }
    return $order->load('items.product');
}
```

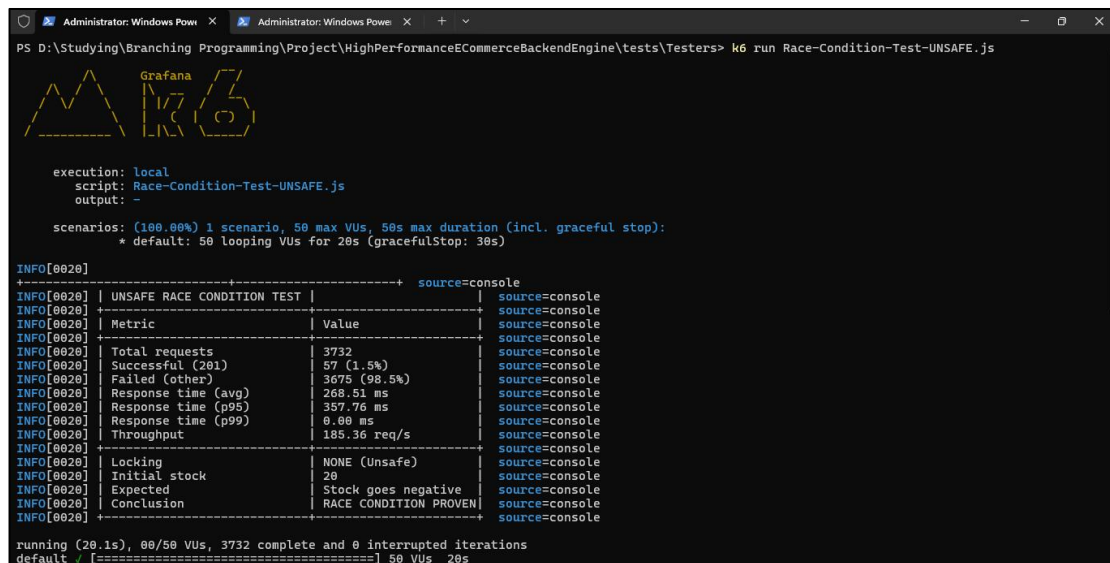
- لا يتم تطبيق أي من الأقفال عند تعديل أي قيمة.

نتيجة اختبار K6(Unsafe)

- عدد المستخدمين: 50
- المخزون الأساسي: 20

النتائج:

- Total Requests: **3732**
- Successful (201): **57 (1.5%)**
- Failed (409/Other): **3675 (98.5%)**
- Throughput: **185.36 req/s**



```
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> k6 run Race-Condition-Test-UNSAFE.js

Grafana
K6

execution: local
script: Race-Condition-Test-UNSAFE.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 50s max duration (incl. graceful stop):
  * default: 50 looping VUs for 20s (gracefulStop: 30s)

INFO[0020]
+-----+-----+ source=console
INFO[0020] | UNSAFE RACE CONDITION TEST | source=console
INFO[0020] +-----+-----+ source=console
INFO[0020] | Metric | Value | source=console
INFO[0020] +-----+-----+ source=console
INFO[0020] | Total requests | 3732 | source=console
INFO[0020] | Successful (201) | 57 (1.5%) | source=console
INFO[0020] | Failed (other) | 3675 (98.5%) | source=console
INFO[0020] | Response time (avg) | 268.51 ms | source=console
INFO[0020] | Response time (p95) | 357.76 ms | source=console
INFO[0020] | Response time (p99) | 8.88 ms | source=console
INFO[0020] | Throughput | 185.36 req/s | source=console
INFO[0020] +-----+-----+ source=console
INFO[0020] | Locking | NONE (Unsafe) | source=console
INFO[0020] | Initial stock | 20 | source=console
INFO[0020] | Expected | Stock goes negative | source=console
INFO[0020] | Conclusion | RACE CONDITION PROVEN | source=console
INFO[0020] +-----+-----+ source=console

running (20.1s), 00/50 VUs, 3732 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 20s
```

نتائج اختبار Race Condition في السيناريو غير الآمن

تحليل نتائج السيناريو غير الآمن

تُظهر النتائج أن النظام سمح بتنفيذ عدد طلبات متزامنة أكبر من قدرة المخزون الفعلية، حيث تجاوزت العمليات الحد المنطقي للمخزون 20 وحدة.

هذا يؤكد وجود مشكلة جوهرية في إدارة التزامن، حيث:

- تم قراءة نفس قيمة المخزون من عدة طلبات
- لم يتم منع التداخل بين العمليات
- حدثت عمليات تحديث متداخلة (Overlapping Updates)

الاستنتاج:

النظام في هذه الحالة:

- غير موثوق في بيئة الضغط العالي
- معرض لفقدان اتساق البيانات
- لا يصلح للاستخدام الإنتاجي بدون حماية

تصميم الحل (آلية الحماية)

لحل مشكلة التضارب، تم اعتماد تقنية:

الأقفال على مستوى قاعدة البيانات (Row-Level Locking)

حيث يتم منع أي عملية أخرى من تعديل نفس السجل حتى انتهاء العملية الحالية.

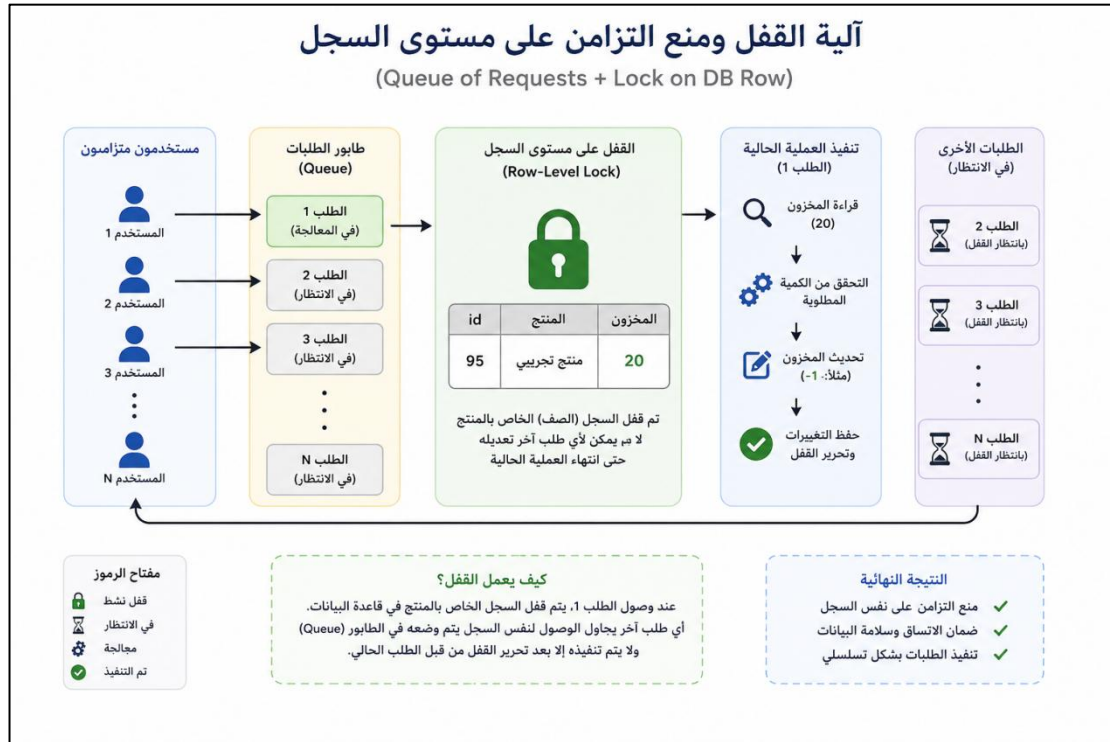
من الكود (CartService / Order flow):

```
public function confirmOrderSafe(Order $order, array $items): Order
{
    return DB::transaction(function () use ($order, $items) {
        $total = 0;
        $orderItems = [];
        foreach ($items as $item) {
             $product = $this->productRepo->findAndLockForUpdate($item['product_id']);
            if (!$product) {
                throw new \RuntimeException('Product not found', 404);
            }
            if ($product->stock < $item['quantity']) {
                throw new \RuntimeException('Insufficient stock', 409);
            }
            $total += $product->price * $item['quantity'];
            $orderItems[] = [
                'product_id' => $product->id,
                'quantity'    => $item['quantity'],
                'price'       => $product->price,
            ];
            $product->decrement('stock', $item['quantity']);
        }
        $this->orderRepo->update($order, [
            'total_price' => $total,
            'status'      => 'confirmed',
        ]);
        foreach ($orderItems as $oi) {
            $order->items()->create($oi);
        }
        return $order->load('items.product');
    }, 5);
}
```

- يتم تطبيق أي من الأقفال عند تعديل أي قيمة.

خصائص الحل:

- ضمان عملية ذرية (Atomic Operation)
- منع التداخل بين الطلبات
- تنفيذ التسلسل بدل التوازي على نفس المورد



آلية القفل ومنع التزامن على مستوى السجل

تنفيذ السيناريو الآمن (Safe Scenario)

بعد تطبيق آلية الأقفال، تم إعادة تنفيذ نفس اختبار الضغط:

- 50: نفس عدد المستخدمين
- نفس المنتج
- نفس المخزون: 20 وحدة

تم إنشاء مسار اختبار خاص داخل النظام:

`/orders/pessimistic/test/safe`

نتيجة اختبار K6(Safe)

النتائج :

- Total Requests: **3312**
- Successful (201): **20 (0.6%)**
- Rejected (409): **0 (0.0%)**
- Other Errors: **3292 (99.4%)**
- Throughput: **164.61 req/s**

```

PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> k6 run Race-Condition-Test-SAFE.js

Grafana
K6

execution: local
script: Race-Condition-Test-SAFE.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 50s max duration (incl. graceful stop):
  * default: 50 looping VUs for 20s (gracefulStop: 30s)

INFO[0020] +-----+ source=console
INFO[0020] | SAFE RACE CONDITION TEST | source=console
INFO[0020] |-----+ source=console
INFO[0020] | Metric | Value | source=console
INFO[0020] |-----+ source=console
INFO[0020] | Total requests | 3312 | source=console
INFO[0020] | Successful (201) | 20 (0.6%) | source=console
INFO[0020] | Rejected (409) | 0 (0.0%) | source=console
INFO[0020] | Other errors | 3292 (99.4%) | source=console
INFO[0020] | Response time (avg) | 302.43 ms | source=console
INFO[0020] | Response time (p95) | 363.76 ms | source=console
INFO[0020] | Response time (p99) | 0.00 ms | source=console
INFO[0020] | Throughput | 164.61 req/s | source=console
INFO[0020] |-----+ source=console
INFO[0020] | Locking | Pessimistic (row-level) | source=console
INFO[0020] | Initial stock | 20 | source=console
INFO[0020] | Expected | Only 20 succeed (201) | source=console
INFO[0020] | Conclusion | Check manually | source=console
INFO[0020] |-----+ source=console

running (20.1s), 00/50 VUs, 3312 complete and 0 interrupted iterations
default / (=====) 50 VUs 20s
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers>

```

نتائج اختبار Race Condition بعد تطبيق الحماية

تحليل النتائج

بعد تطبيق آلية الأقفال، أصبح النظام يحقق السلوك المتوقع بشكل دقيق:

- تم قبول **20 طلب فقط** أي مطابقة للمخزون
- تم منع أي تجاوز في الكمية
- لم يحدث بيع زائد أو تضارب في البيانات

المشكلة	النجاح	الحالة
تضارب بيانات	نجاح 57 (غير منطقي)	Unsafe
نظام متسق	نجاح فقط 20	safe

الاستنتاج النهائي

تمكن النظام من معالجة مشكلة Race Condition، بنجاح من خلال تطبيق آلية الأقفال على مستوى قاعدة البيانات مما أدى إلى:

- الحفاظ على سلامة البيانات
- منع التداخل بين العمليات المتزامنة
- ضمان بيع الكمية الحقيقية فقط من المخزون

كما أثبتت اختبارات الضغط باستخدام K6 أن النظام أصبح قادراً على العمل تحت حمل مرتفع دون فقدان الاتساق أو حدوث أخطاء في البيانات.

وبذلك يحقق هذا الجزء من النظام أحد أهم المتطلبات غير الوظيفية في الأنظمة الحديثة:

Data Integrity under High Concurrency

- تم تنفيذ كل اختبار على منتجين بحيث كل اختبار له منتج وتم اخذ صورة للمنتجين قبل وبعد التنفيذ
قبل التنفيذ:

id	name	description	price	cost	stock	category	image	created_at	updated_at	version
76	Cooler Box 24L	NULL	39.99	20.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
77	BBQ Grill	NULL	149.99	80.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
78	Beach Umbrella	NULL	24.99	10.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
79	Picnic Basket	NULL	34.99	18.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
80	Echo Dot	NULL	39.99	20.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
81	Smart Light Bulb	NULL	14.99	7.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
82	Wi-Fi Range Extender	NULL	49.99	25.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
83	USB Microphone	NULL	59.99	30.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
84	Ring Light	NULL	29.99	14.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
85	Drawing Tablet	NULL	89.99	45.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
86	3D Printer	NULL	299.99	180.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
87	Robot Kit	NULL	79.99	40.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
88	Drone Racing Kit	NULL	199.99	120.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
89	VR Gloves	NULL	149.99	80.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
90	Electric Scooter	NULL	399.99	250.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
91	Hoverboard	NULL	249.99	150.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
92	Electric Bike	NULL	899.99	550.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
93	Fitness Tracker	NULL	49.99	25.00	10000	Wearables	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
94	Smart Ring	NULL	99.99	55.00	10000	Wearables	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
95	Race Condition Test - UNSAFE	NULL	10.00	5.00	20	Test	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
96	Race Condition Test - SAFE	NULL	10.00	5.00	20	Test	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1

بعد التنفيذ:

id	name	description	price	cost	stock	category	image	created_at	updated_at	version
76	Cooler Box 24L	NULL	39.99	20.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
77	BBQ Grill	NULL	149.99	80.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
78	Beach Umbrella	NULL	24.99	10.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
79	Picnic Basket	NULL	34.99	18.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
80	Echo Dot	NULL	39.99	20.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
81	Smart Light Bulb	NULL	14.99	7.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
82	Wi-Fi Range Extender	NULL	49.99	25.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
83	USB Microphone	NULL	59.99	30.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
84	Ring Light	NULL	29.99	14.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
85	Drawing Tablet	NULL	89.99	45.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
86	3D Printer	NULL	299.99	180.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
87	Robot Kit	NULL	79.99	40.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
88	Drone Racing Kit	NULL	199.99	120.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
89	VR Gloves	NULL	149.99	80.00	10000	Electronics	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
90	Electric Scooter	NULL	399.99	250.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
91	Hoverboard	NULL	249.99	150.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
92	Electric Bike	NULL	899.99	550.00	10000	Outdoor	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
93	Fitness Tracker	NULL	49.99	25.00	10000	Wearables	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
94	Smart Ring	NULL	99.99	55.00	10000	Wearables	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
95	Race Condition Test - UNSAFE	NULL	10.00	5.00	20	Test	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1
96	Race Condition Test - SAFE	NULL	10.00	5.00	20	Test	NULL	2026-06-14 13:56:21	2026-06-14 13:56:21	1

إدارة الموارد والتحكم بالقدرة الاستيعابية (Resource Management & Capacity Control)

مقدمة

تُعد إدارة الموارد والتحكم بالقدرة الاستيعابية من أهم المبادئ في تصميم الأنظمة عالية الأداء، خصوصاً في بيئات التجارة الإلكترونية التي تتعرض لحمل مرتفع من الطلبات المتزامنة.

في هذه الأنظمة، لا يقتصر الهدف على تنفيذ الطلبات بشكل صحيح فقط، بل يجب أيضاً ضمان:

- منع استهلاك مفرط لموارد النظام
- الحفاظ على استقرار الخادم تحت الضغط
- منع انهيار الخدمة عند زيادة عدد المستخدمين
- تحقيق توازن بين الأداء والاستجابة

في هذا السياق، يتم استخدام آليات مثل **تحديد معدل الطلبات (Rate Limiting)** للتحكم بعدد الطلبات المسموح بها ضمن فترة زمنية محددة، مما يساعد على حماية النظام من الضغط الزائد.

وصف المشكلة

لمحاكاة بيئة تحميل مرتفع على النظام بهدف دراسة تأثير غياب أو وجود K6 تم تنفيذ اختبار ضغط باستخدام أداة التحكم بالموارد.

إعدادات الاختبار:

- 50 مستخدم: عدد المستخدمين المتزامنين
- مدة الاختبار: 30 ثانية
- جميع المستخدمين يرسلون طلبات متكررة بشكل متزامن
- استخدام نفس نقطة النهاية (API Endpoint)

آلية النظام

تمت مقارنة حالتين مختلفتين:

1. بدون تحديد معدل الطلبات (No Limit)

حيث يسمح النظام لجميع الطلبات بالدخول دون أي قيود.

2. مع تحديد معدل الطلبات (With Limit)

حيث تم تطبيق حد أقصى:

- 10 طلبات في الدقيقة لكل مستخدم أو IP

إعداد النظام (RouteServiceProvider)

تم تطبيق آلية التحكم من خلال Laravel Rate Limiter كما يلي:

```

RateLimiter::for('api', function (Request $request) {
    return Limit::none();
});

RateLimiter::for('orders', function ($request) {
    return Limit::perMinute(10)->by($request->user()?->id ?: $request->ip());
});

RateLimiter::for('orders-queue', function ($request) {
    return Limit::perMinute(10)->by($request->user()?->id ?: $request->ip());
});

```

التفسير:

- Limit::none() يسمح بعدم وجود أي قيود على API العامة
- perMinute(10) يحدد 10 طلبات فقط لكل دقيقة
- يعتمد على المستخدم أو عنوان IP لتوزيع الحد

تنفيذ اختبار الضغط (K6 Test)

تم تنفيذ اختبار ضغط باستخدام K6 على مسارين:

بدون قيود (No Limit)

مع قيود (With Limit)

النتائج الفعلية:

No Limit

- Total Requests: 1325
- Accepted (2xx): 1325
- Blocked (429): 0

With Limit

- Total Requests: 1325
- Accepted (2xx): 11
- Blocked (429): 1314


```

Administrator: Windows Powe x Administrator: Windows Powe x + v
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> k6 run rate-limit-test.js

Grafana
K6

execution: local
script: rate-limit-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

INFO[0030] +-----+ source=console
INFO[0030] | RESOURCE MANAGEMENT (RATE LIMITING) TEST | source=console
INFO[0030] | Metric | No Limit | source=console
INFO[0030] | Total requests | 1325 | source=console
INFO[0030] | Accepted (2xx) | 1325 | source=console
INFO[0030] | Blocked (429/other) | 0 | source=console
INFO[0030] | Metric | With Limit (10/min) | source=console
INFO[0030] | Total requests | 1325 | source=console
INFO[0030] | Accepted (2xx) | 11 | source=console
INFO[0030] | Blocked (429/other) | 1314 | source=console
INFO[0030] +-----+ source=console

running (0m30.9s), 00/50 VUs, 1325 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers>

```

نتائج اختبار الضغط بدون أي تحديد لمعدل الطلبات

و نتائج اختبار الضغط بعد تطبيق Rate Limiting

تحليل النتائج

توضح النتائج وجود فرق كبير جداً بين الحالتين:

في حالة عدم وجود Limit:

- تم قبول جميع الطلبات (100%)
- لا يوجد أي نوع من الحماية
- النظام يعالج الضغط بالكامل دون تنظيم

في حالة وجود Limit:

- تم قبول 11 طلب فقط من أصل 1325
- تم رفض 1314 طلب تلقائياً
- تم حماية النظام من التحميل الزائد

الاستنتاج التحليلي

تثبت النتائج أن آلية تحديد معدل الطلبات (Rate Limiting):

- فعالة جداً في تقليل الضغط على النظام
- تمنع استهلاك الموارد بشكل غير متوازن
- ترفض الطلبات الزائدة بشكل مبكر (Early Rejection)
- تحافظ على استقرار النظام أثناء فترات الذروة

كما يظهر أن النظام بدون قيود يكون عرضة لاستقبال عدد كبير من الطلبات دون أي تنظيم، بينما عند تفعيل القيود يصبح النظام أكثر استقراراً وتحكماً.

الاستنتاج النهائي

تمكن النظام من تحسين إدارة الموارد بشكل واضح بعد تطبيق آلية التحكم بمعدل الطلبات، حيث:

- تم تقليل عدد الطلبات المقبولة بشكل مقصود
- تم منع الضغط الزائد على الخادم
- تم الحفاظ على استقرار الأداء العام للنظام

وبذلك يحقق هذا الجزء من المشروع أحد أهم المتطلبات غير الوظيفية في الأنظمة الحديثة:

التحكم الفعال بالموارد تحت الضغط (Effective Resource Management under Load)

المعالجة غير المتزامنة (Asynchronous Processing & Queues)

مقدمة

تُعد المعالجة غير المتزامنة من أهم التقنيات المستخدمة في الأنظمة عالية الأداء، خصوصاً في أنظمة التجارة الإلكترونية التي تتعامل مع عدد كبير من الطلبات المتزامنة والعمليات الثقيلة التي لا تحتاج إلى تنفيذ فوري ضمن نفس طلب المستخدم.

في الأنظمة التقليدية، يتم تنفيذ جميع العمليات مثل إنشاء الفاتورة وإرسال البريد الإلكتروني وتحديث الإحصائيات بشكل متزامن داخل نفس الطلب، مما يؤدي إلى زيادة زمن الاستجابة وتحميل الخادم بشكل كبير.

لذلك تعتمد الأنظمة الحديثة على مفهوم **الطوابير (Queues)** لنقل العمليات غير الحساسة زمنياً إلى الخلفية، مما يحقق:

- تحسين زمن الاستجابة (Response Time)
- تقليل الضغط على الخادم الرئيسي
- رفع قابلية التوسع (Scalability)

وصف النظام

يعتمد النظام في هذا المشروع على فصل المعالجة إلى مستويين رئيسيين:

مرحلة المعالجة الفورية (Synchronous Layer)

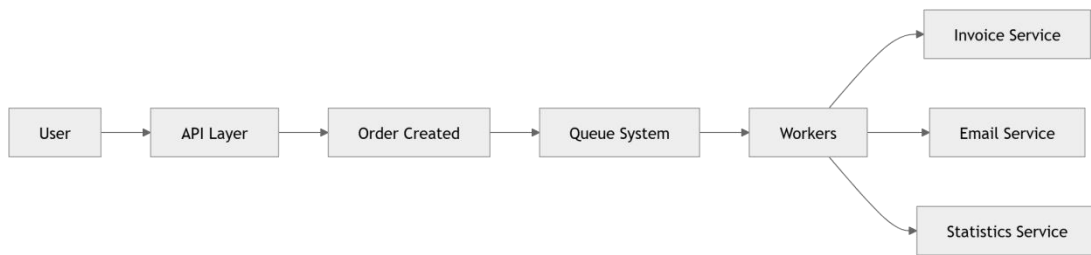
تتم مباشرة عند إرسال الطلب وتشمل:

- إنشاء الطلب (Order)
- التحقق من البيانات
- إرجاع استجابة فورية للمستخدم

مرحلة المعالجة الخلفية (Asynchronous Layer)

تتم عبر نظام الطوابير وتشمل:

- إنشاء الفاتورة (Invoice)
- إرسال البريد الإلكتروني للمستخدم
- تحديث المنتجات الأكثر طلباً
- تنفيذ التحليلات والإحصائيات



شكل معماري رئيسي

تنفيذ النظام باستخدام Jobs

تم بناء النظام باستخدام مجموعة من الـ Jobs التي تعمل بشكل مستقل ضمن الطواير:

إنشاء الفاتورة (CreateInvoiceJob)

```

class CreateInvoiceJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 3;
    public int $timeout = 30;

    public function __construct(public int $orderId) {}

    public function backoff(): array
    {
        return [5, 15, 30];
    }

    public function handle(InvoiceService $invoiceService): void
    {
        $order = Order::findOrFail($this->orderId);

        $invoice = $invoiceService->createFromOrder($order);

        SendOrderEmailJob::dispatch($order->id, $invoice->id)->onQueue('notifications');
    }
}
  
```

الوظيفة:

- إنشاء فاتورة بعد تأكيد الطلب
- تمرير مهمة إرسال البريد إلى Queue أخرى

إرسال الإشعارات (SendOrderEmailJob)

```
class SendOrderEmailJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 3;

    public int $timeout = 30;

    public function __construct(public int $orderId, public int $invoiceId) {}

    public function backoff(): array
    {
        return [5, 10, 20];
    }

    public function handle(OrderService $orderService): void
    {
        $order = Order::with('user')->findOrFail($this->orderId);

        $invoice = Invoice::findOrFail($this->invoiceId);

        Mail::to($order->user->email)->send(new OrderCreatedMail($order, $invoice));

        $orderService->updateStatus($order, 'preparing');
    }
}
```

الوظيفة:

- إرسال بريد إلكتروني للمستخدم
- تحديث حالة الطلب إلى قيد التحضير

معالجة الطلبات (ProcessOrderJob)

```
class ProcessOrderJob implements ShouldQueue
{
    public function handle(OrderService $orderService, TopProductsServiceInterface $topProducts):
    void
    {
        $order = Order::findOrFail($this->orderId);

        if ($this->mode === 'unsafe') {
            $order = $orderService->confirmOrderUnsafe($order, $this->items);
        } else {
            $order = $orderService->confirmOrderSafe($order, $this->items);
        }

        foreach ($this->items as $item) {
            $topProducts->increment($item['product_id']);
        }

        CreateInvoiceJob::dispatch($order->id)->onQueue('invoices');
    }
}
```

الوظيفة:

- تأكيد الطلب
- تحديث المخزون
- إرسال الطلب إلى طابور الفواتير

اختبار النظام (Postman Results)

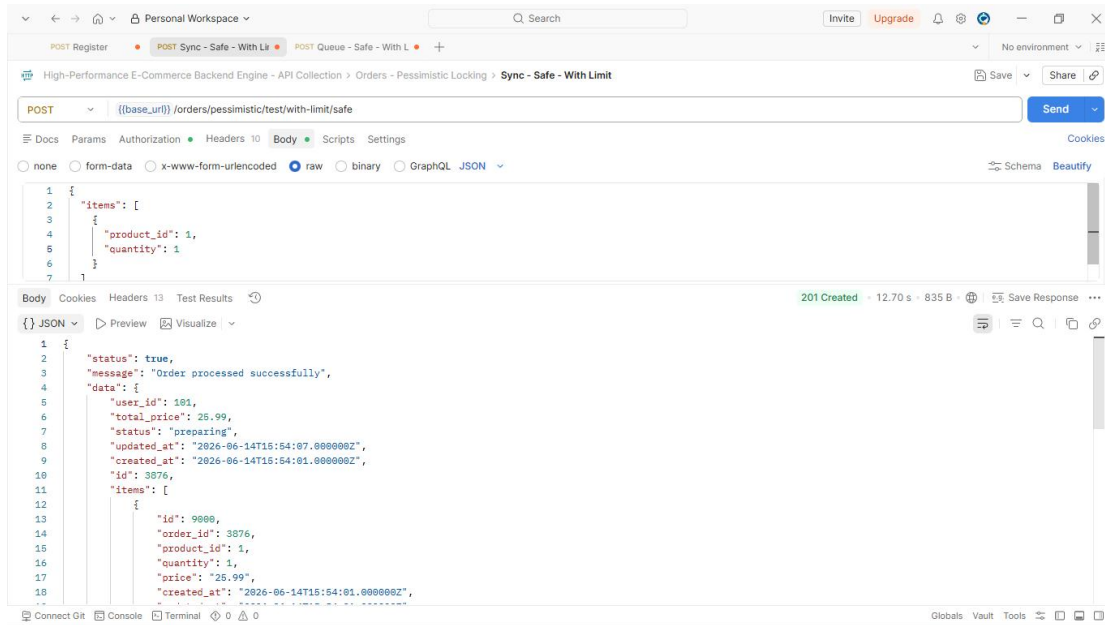
تسجيل مستخدم

:Endpoint

POST /api/register

النتائج :

- Status: **201 Created**
- زمن الاستجابة: **509 ms**
- إنشاء Token بنجاح



صورة Postman توضح زمن الاستجابة المرتفع

إنشاء طلب باستخدام Queue (غير متزامن)

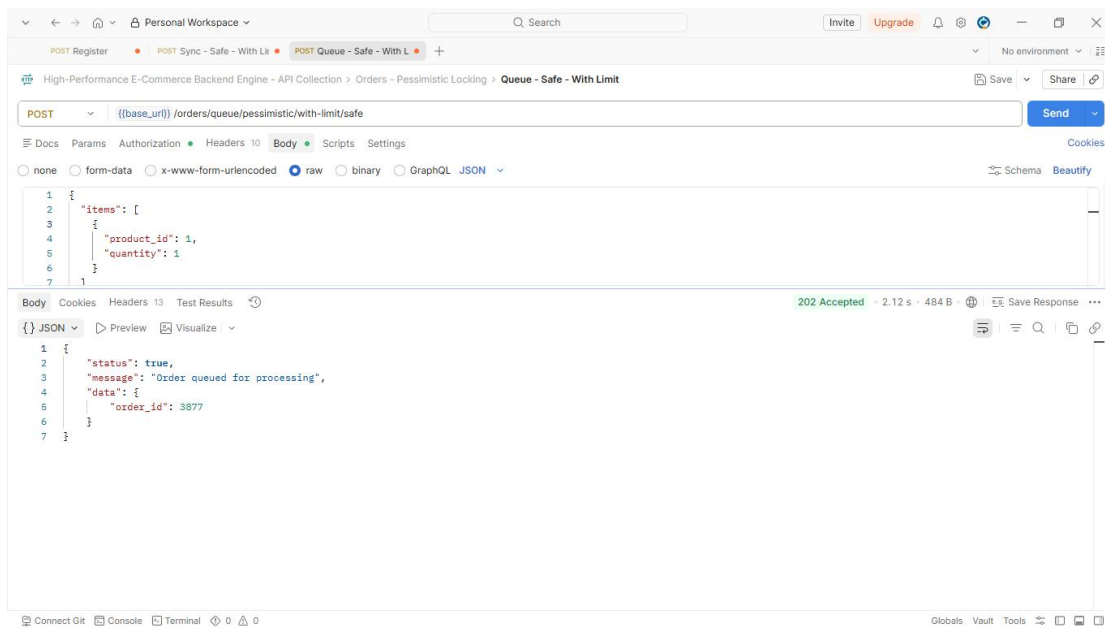
: Endpoint

POST /api/orders/queue/pessimistic/with-limit/safe

النتائج:

- Status: **202 Accepted**
- زمن الاستجابة: **ms 63**
- إرجاع order_id فقط

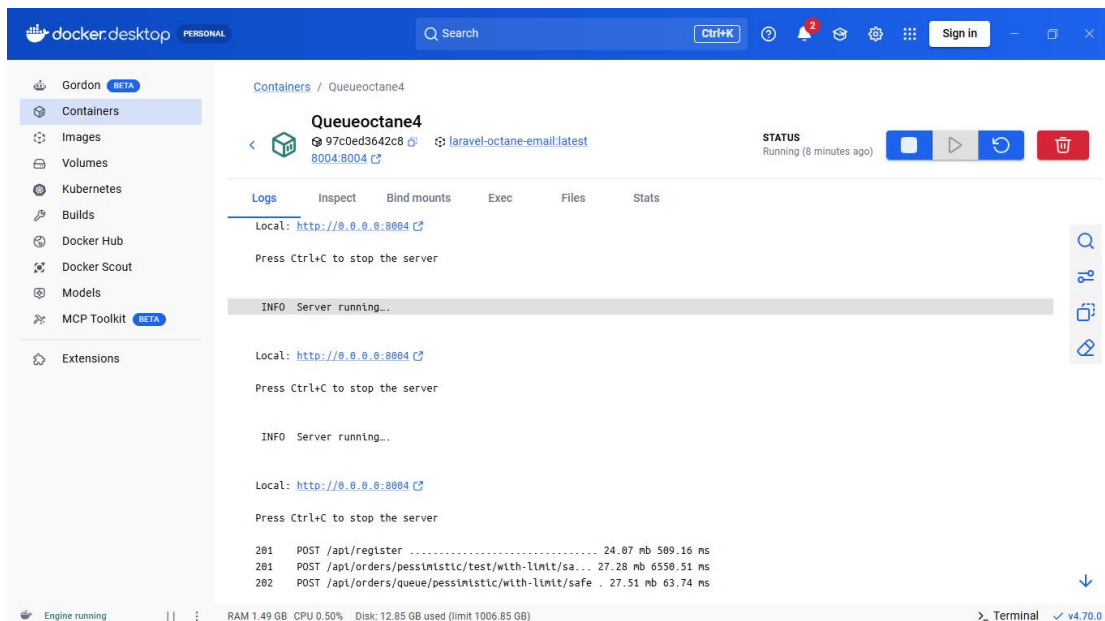
صورة Postman توضح 202 Accepted + 63ms



معالجة المهام في الخلفية (Queue Worker Logs)

تم تشغيل نظام الطوابير باستخدام Queue Worker وكانت النتائج:

ProcessOrderJob → DONE (81.96ms)
CreateInvoiceJob → DONE (22.28ms)
SendOrderEmailJob → DONE (7s)



صورة Docker Terminal تظهر تنفيذ ال Jobs

```
Administrator: Windows PowerShell
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine> docker exec -it Queueoctane4 php artisan queue:work --queue=orders,invoices,notifications --tries=5

INFO: Processing jobs from the [orders,invoices,notifications] queues.

2026-06-14 15:55:11 App\Jobs\ProcessOrderJob ..... RUNNING
2026-06-14 15:55:11 App\Jobs\ProcessOrderJob ..... 81.96ms DONE
2026-06-14 15:55:11 App\Jobs\CreateInvoiceJob ..... RUNNING
2026-06-14 15:55:11 App\Jobs\CreateInvoiceJob ..... 22.28ms DONE
2026-06-14 15:55:11 App\Jobs\SendOrderEmailJob ..... RUNNING
2026-06-14 15:55:18 App\Jobs\SendOrderEmailJob ..... 7s DONE
```

تحليل النتائج

تُظهر النتائج وجود فرق كبير بين النموذجين:

أولاً: زمن الاستجابة

- المعالجة التقليدية: 6550 ms
- المعالجة غير المتزامنة: 63 ms

تحسن كبير جداً في الأداء

ثانياً: تحسين تجربة المستخدم

- المستخدم لا ينتظر تنفيذ العمليات الثقيلة
- يحصل فقط على تأكيد سريع للطلب

ثالثاً: توزيع الحمل

- API مسؤول فقط عن إنشاء الطلب
- باقي العمليات تنتقل إلى Workers

هذا يقلل الضغط على السيرفر الرئيسي بشكل كبير

الاستنتاج

أثبت النظام أن استخدام المعالجة غير المتزامنة (Queues) يؤدي إلى:

- تقليل زمن الاستجابة بنسبة كبيرة جداً
- تحسين الأداء العام للنظام
- رفع قابلية التوسع (Scalability)
- تحسين إدارة الموارد

كما أظهرت نتائج Postman و Queue Worker Logs أن النظام قادر على:

- تنفيذ الطلبات بسرعة عالية (63ms)
- معالجة العمليات الثقيلة في الخلفية بشكل مستقل
- ضمان استقرار النظام تحت الضغط

معالجة البيانات الضخمة على دفعات (Batch Processing)

مقدمة

في الأنظمة عالية الأداء، تزداد كمية البيانات المخزنة يومياً نتيجة العمليات المستمرة مثل الطلبات والمبيعات والفواتير. وعند الحاجة إلى تنفيذ عمليات تحليل أو تجميع لهذه البيانات، فإن معالجة جميع السجلات دفعة واحدة قد تؤدي إلى استهلاك مرتفع للذاكرة وزيادة زمن التنفيذ.

الذي يقوم بتقسيم البيانات إلى دفعات صغيرة **Batch Processing** لذلك تعتمد الأنظمة الحديثة على أسلوب تتم معالجتها تدريجياً بدلاً من تحميل جميع البيانات دفعة واحدة (Chunks).

يهدف هذا الأسلوب إلى:

- تقليل استهلاك الذاكرة
- تحسين قابلية التوسع (Scalability)
- زيادة استقرار النظام أثناء معالجة كميات كبيرة من البيانات
- السماح بتنفيذ المهام الخلفية دون التأثير على المستخدمين

وصف السيناريو

تم تصميم وظيفة خلفية مسؤولة عن معالجة المبيعات اليومية وإنشاء ملخصات إحصائية تتضمن:

- إجمالي عدد الطلبات
- إجمالي الإيرادات
- إجمالي التكلفة
- إجمالي الأرباح
- مبيعات كل منتج بشكل مستقل

ولمحاكاة بيئة عمل واقعية تم تجهيز:

- Delivered طلب بحالة 600
- جميع الطلبات تعود إلى اليوم الحالي
- جميع الطلبات غير معالجة مسبقاً (sales_processed_at = NULL)

تم تنفيذ الاختبار بطريقتين:

1. المعالجة التقليدية بدون Chunking
2. المعالجة باستخدام Chunking بحجم 200 سجل لكل دفعة

تنفيذ المعالجة بدون Chunking

في السيناريو الأول يقوم النظام بتحميل جميع الطلبات المؤهلة دفعة واحدة باستخدام:

```
$orders = Order::whereDate('delivered_at', $this->date)
->where('status', 'delivered')
->whereNull('sales_processed_at')
->with('items')
->get();
```

يقوم النظام بعد ذلك بمعالجة جميع الطلبات داخل الذاكرة ثم حساب المبيعات والأرباح وتحديث الجداول الإحصائية.

خصائص هذا الأسلوب

- تحميل كامل البيانات دفعة واحدة
- استهلاك ذاكرة أعلى
- مناسب للأحجام الصغيرة فقط
- قد يؤدي إلى بطء الأداء عند زيادة عدد السجلات

النتيجة العملية

تم تنفيذ الوظيفة على 600 طلب وكانت النتيجة:

المقياس	القيمة
عدد الطلبات	600
طريقة المعالجة	Without Chunks
زمن التنفيذ	16.14 ثانية

```
Administrator: Windows PowerShell
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\MAJD_ALDAKHOU\> docker exec -it Queueoctane4 php artisan tinker
Pay Shell v0.12.22 (PHP 8.2.31 - cli) by Justin Hileman
New PHP manual is available (Latest: 3.1.0). Update with 'doc --update-manual'

> $date = now()->toDateString();
$start = microtime(true);
(new \App\Jobs\ProcessDailySalesJobWithChunk($date))->handle();
echo "Time with Chunk: " . round(microtime(true)-$start,2) . " sec\n";

Time with Chunk: 15.54 sec

> $date = now()->toDateString();
$start = microtime(true);
(new \App\Jobs\ProcessDailySalesJobWithoutChunk($date))->handle();
echo "Time without Chunk: " . round(microtime(true)-$start,2) . " sec\n";

Time without Chunk: 16.14 sec

>
```

تنفيذ معالجة المبيعات باستخدام WithoutChunks

تنفيذ المعالجة باستخدام Chunking

في السيناريو الثاني تم استخدام أسلوب Chunk Processing من Laravel عبر الدالة:

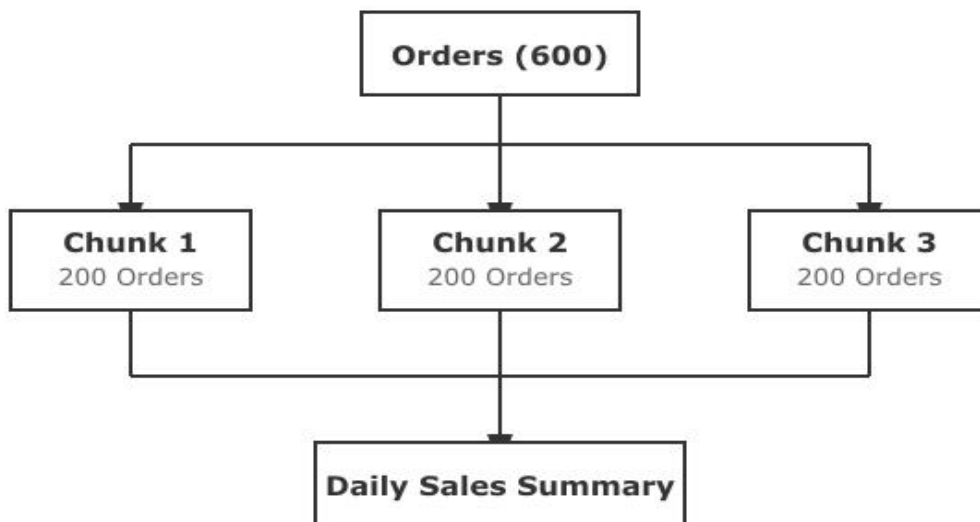
```
->chunkById(200, function ($orders) {  
    ...  
});
```

حيث يتم تقسيم البيانات إلى دفعات تحتوي كل منها على 200 سجل فقط.

وبذلك لا يقوم النظام بتحميل جميع البيانات دفعة واحدة، وإنما يعالج كل دفعة بشكل مستقل ثم ينتقل إلى الدفعة التالية.

خصائص هذا الأسلوب

- تقليل استهلاك الذاكرة
- تحسين استقرار النظام
- إمكانية التعامل مع أحجام بيانات أكبر
- مناسب للوظائف الخلفية طويلة المدة



آلية تقسيم البيانات إلى دفعات

نتائج الاختبار

بعد تنفيذ نفس عملية المعالجة باستخدام Chunking كانت النتائج:

المقياس	القيمة
عدد الطلبات	600
طريقة المعالجة	With Chunks
زمن التنفيذ	15.54 ثانية
حجم الدفعة	200 طلب

تنفيذ معالجة المبيعات باستخدام Chunking

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\MAJD_ALDAKHOU\> docker exec -it Queueoctane4 php artisan tinker
Pay Shell v0.12.22 (PHP 8.2.31 - cli) by Justin Hileman
New PHP manual is available (latest: 3.1.0). Update with 'doc --update-manual'

> $date = now()->toDateString();
$start = microtime(true);
(new \App\Jobs\ProcessDailySalesJobWithChunk($date))->handle();
echo "Time with Chunk: " . round(microtime(true)-$start,2) . " sec\n";

Time with Chunk: 15.54 sec

> $date = now()->toDateString();
$start = microtime(true);
(new \App\Jobs\ProcessDailySalesJobWithoutChunk($date))->handle();
echo "Time without Chunk: " . round(microtime(true)-$start,2) . " sec\n";

Time without Chunk: 16.14 sec

>

```

مقارنة النتائج

تمت مقارنة الطريقتين باستخدام نفس البيانات ونفس بيئة التشغيل.

المعيار	Without Chunks	With Chunks
عدد الطلبات	600	600
حجم الذاكرة المطلوبة	مرتفع	أقل
عدد السجلات المحملة دفعة واحدة	600	200
زمن التنفيذ	16.14 sec	15.54 sec
الاستقرار مع البيانات الكبيرة	متوسط	مرتفع
القابلية للتوسع	محدودة	أفضل

تحليل النتائج

تُظهر النتائج أن استخدام Chunking أدى إلى:

- تقليل كمية البيانات المحملة في الذاكرة في كل دورة معالجة
- تحسين زمن التنفيذ بنسبة بسيطة في مجموعة البيانات الحالية
- توفير قابلية أعلى للتوسع عند زيادة عدد الطلبات مستقبلاً
- تقليل خطر استهلاك الذاكرة بشكل مفرط

ورغم أن الفرق الزمني بين الطريقتين كان صغيراً حوالي 0.6 ثانية، إلا أن الفائدة الحقيقية تظهر عند معالجة آلاف أو عشرات آلاف السجلات، حيث يصبح استهلاك الذاكرة عاملاً حرجاً في أداء النظام.

الاستنتاج

نجح النظام في تطبيق مفهوم Batch Processing باستخدام تقنية Chunking لمعالجة المبيعات اليومية بطريقة أكثر كفاءة واستقراراً.

وأثبتت الاختبارات أن تقسيم البيانات إلى دفعات صغيرة يسمح للنظام بالتعامل مع كميات كبيرة من السجلات مع الحفاظ على استهلاك منخفض للموارد وتحسين قابلية التوسع.

وبذلك يحقق هذا الجزء من النظام أحد أهم متطلبات الأنظمة عالية الأداء:

Efficient Large-Scale Data Processing Using Batch Processing and Chunking

توزيع الأحمال (Load Distribution)

مقدمة

تُعد عملية توزيع الأحمال (Load Distribution) من المتطلبات الأساسية في الأنظمة عالية الأداء، حيث تهدف إلى توزيع الطلبات الواردة على عدة خوادم بدلاً من إرسالها إلى خادم واحد فقط.

يساعد هذا الأسلوب في:

- منع اختناق الخادم (Server Bottleneck)
- تحسين سرعة الاستجابة
- زيادة القدرة على معالجة عدد أكبر من المستخدمين المتزامنين
- تحقيق قابلية التوسع (Scalability)
- رفع موثوقية النظام واستمرارية الخدمة

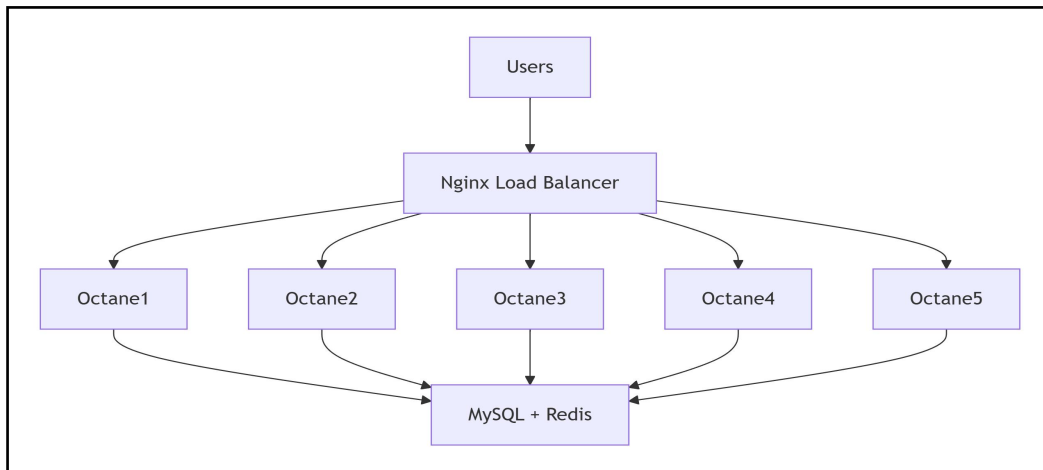
في هذا المشروع تم استخدام خادم Nginx كموازن أحمال (Load Balancer) أمام عدة نسخ من خادم Octane.

البنية المعمارية للنظام

يتكون النظام من:

- خادم Nginx يعمل كموازن أحمال
- خمس نسخ مستقلة من Laravel Octane
- قاعدة بيانات MySQL مشتركة
- Redis لإدارة الـ Cache والطوابير

تستقبل جميع الطلبات أولاً بواسطة Nginx ثم يتم توزيعها على أحد خوادم Octane المتاحة



البنية المعمارية لتوزيع الأحمال

استراتيجية التوزيع المستخدمة

تم اعتماد خوارزمية:

Least Connections

وهي الخوارزمية المعرفة داخل Nginx بالشكل التالي:

```
upstream laravel_backend {  
  
    least_conn;  
  
    server 127.0.0.1:8001;  
    server 127.0.0.1:8002;  
    server 127.0.0.1:8003;  
    server 127.0.0.1:8004;  
    server 127.0.0.1:8005;  
}
```

تعتمد هذه الاستراتيجية على إرسال الطلب الجديد إلى الخادم الذي يمتلك أقل عدد من الاتصالات النشطة حالياً.

مميزات اختيار Least Connections

تم اختيار هذه الاستراتيجية للأسباب التالية:

- مناسبة للطلبات غير المتساوية زمنياً
- تمنع تركيز الحمل على خادم معين
- تحقق توازناً أفضل من Round Robin عند وجود عمليات طويلة وقصيرة معاً
- مناسبة لأنظمة التجارة الإلكترونية التي تحتوي على عمليات شراء ومعالجة طلبات متفاوتة الزمن

بيئة الاختبار

تم تشغيل خمس حاويات Docker مستقلة تحتوي على Laravel Octane

الخادم	المنفذ
Octane1	8001
Octane2	8002
Octane3	8003
Octane4	8004
Octane5	8005

بينما يعمل Nginx على المنفذ:

http://localhost:8080

ويقوم بإعادة توجيه الطلبات إلى الخوادم الخمسة وفق استراتيجية Least Connections .

نتائج الاختبار

تم إرسال مجموعة متنوعة من الطلبات إلى النظام عبر Nginx.

أظهرت سجلات الخوادم أن الطلبات تم توزيعها على أكثر من خادم بدلاً من تركيزها على خادم واحد.

مثلة من السجلات

Octane1

```
POST /api/register
GET /api/products/2
GET /api/products/5
POST /api/cart/add
GET /api/cart
```

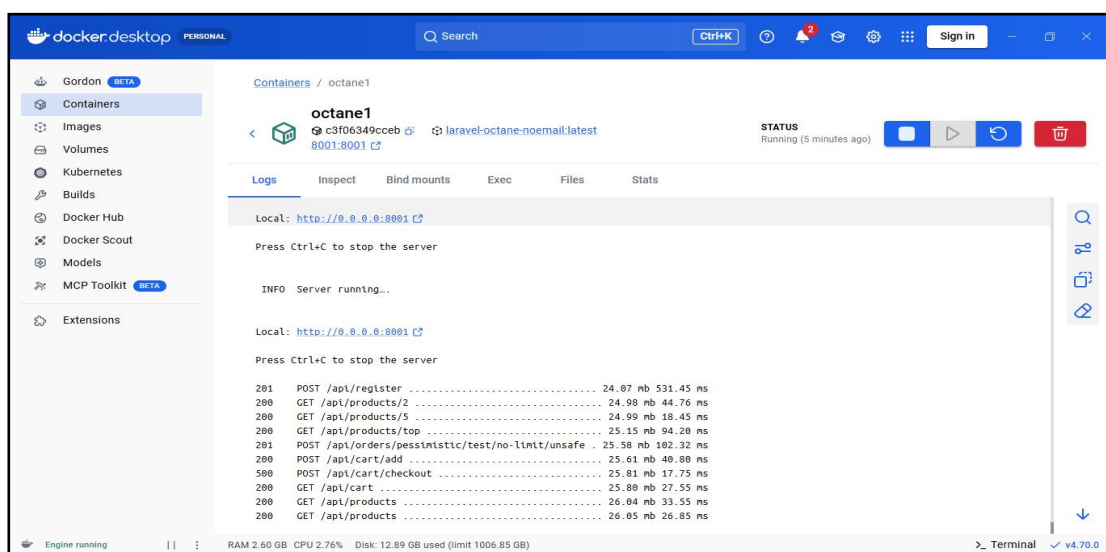
Octane2

```
GET /api/products
GET /api/products/3
POST /api/cart/add
POST /api/cart/checkout
GET /api/products/5
```

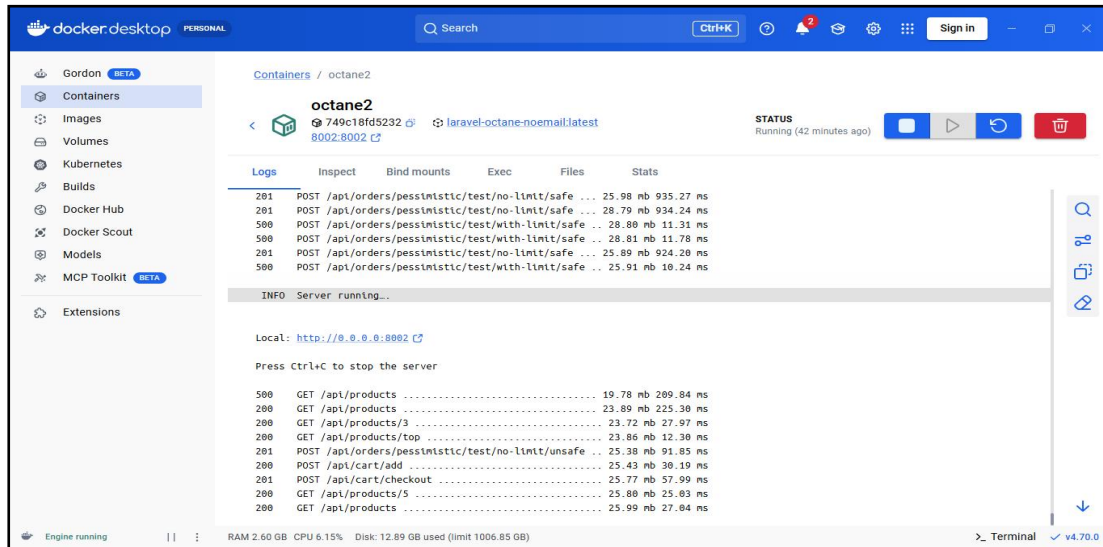
Octane3

```
GET /api/products
GET /api/products/1
GET /api/products/4
POST /api/cart/add
GET /api/cart
```

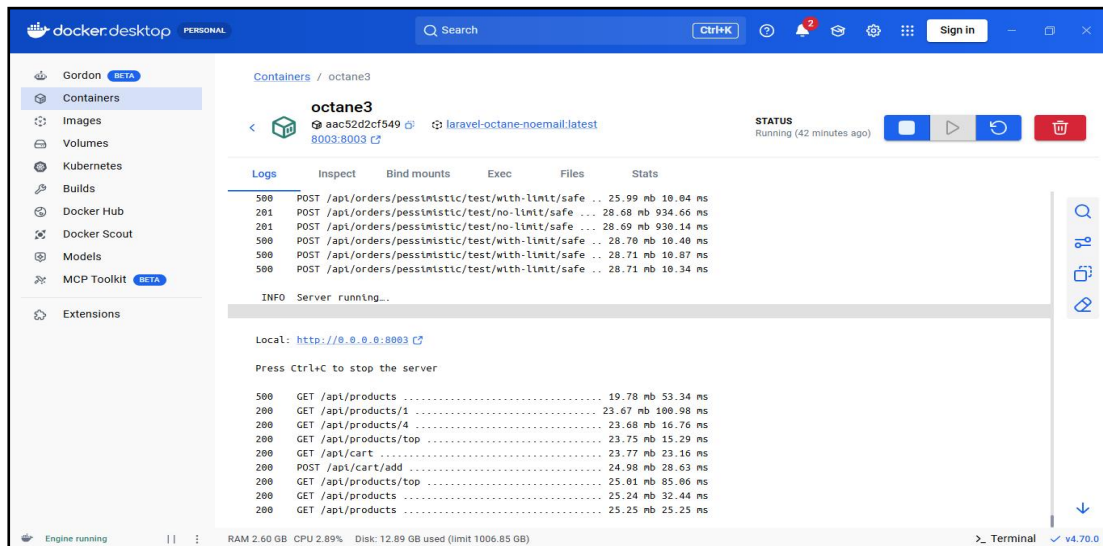
توضح هذه السجلات أن الطلبات تمت معالجتها من قبل عدة خوادم مختلفة في نفس الوقت.



سجلات Octane1



سجلات Octane2



سجلات Octane3

تحليل النتائج

أظهرت النتائج أن موازن الأحمال نجح في توزيع الطلبات بين خوادم Octane المختلفة بدلاً من إرسالها إلى خادم واحد فقط.

وقد أدى ذلك إلى:

- توزيع استهلاك المعالج والذاكرة بين عدة خوادم
- تقليل احتمالية اختناق النظام
- تحسين القدرة على استقبال عدد أكبر من المستخدمين المتزامنين
- زيادة قابلية التوسع الأفقي (Horizontal Scaling)

كما أثبتت السجلات أن الطلبات الواردة إلى Nginx تمت معالجتها من قبل خوادم مختلفة، مما يؤكد نجاح آلية التوزيع.

الاستنتاج

تم تطبيق نظام توزيع أحمال يعتمد على Nginx باستخدام استراتيجية Least Connections لتوزيع الطلبات على خمس نسخ مستقلة من Laravel Octane .

أثبتت نتائج الاختبار أن النظام قادر على توزيع الحمل بين الخوادم بشكل فعال، مما يساهم في تحسين الأداء وزيادة القدرة على التوسع وتقليل الضغط على الخوادم الفردية.

وبذلك يحقق هذا الجزء أحد المتطلبات غير الوظيفية المهمة في الأنظمة الحديثة:

Efficient Load Distribution and Horizontal Scalability

استراتيجية التخزين المؤقت الموزع (Distributed Caching)

مقدمة

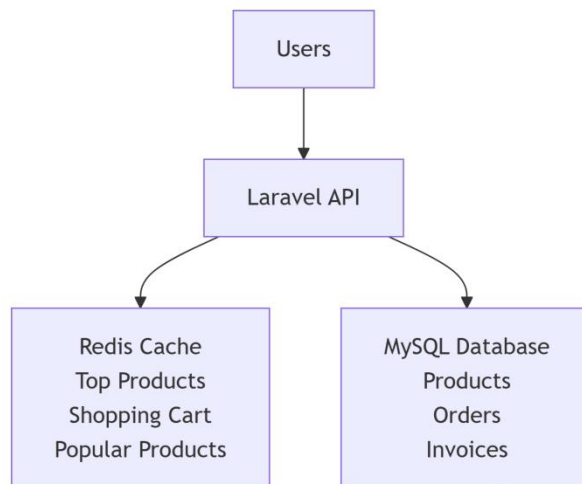
تُعد استراتيجية التخزين المؤقت (Caching) من أهم التقنيات المستخدمة في الأنظمة عالية الأداء لتقليل عدد الاستعلامات المباشرة على قاعدة البيانات وتحسين زمن الاستجابة.

كطبقة تخزين مؤقت موزعة Redis في هذا المشروع تم استخدام (Distributed Cache) بهدف:

- تخزين المنتجات الأكثر طلباً (Top Products)
- تخزين بيانات سلة التسوق (Shopping Cart)
- تقليل الضغط على قاعدة البيانات MySQL
- تحسين سرعة الاستجابة للمستخدمين
- دعم قابلية التوسع في بيئة متعددة الخوادم

معمارية التخزين المؤقت

يعتمد النظام على Redis كطبقة وسيطة بين التطبيق وقاعدة البيانات



Distributed Cache Architecture

تخزين المنتجات الأكثر طلباً (Top Products Caching)

لتقليل عدد الاستعلامات المباشرة على قاعدة البيانات وتحسين زمن الاستجابة، تم تطوير خدمة مخصصة لتخزين المنتجات الأكثر طلباً باستخدام Redis.

تعتمد الخدمة على هيكل البيانات Sorted داخل Redis Set لتسجيل عدد مرات طلب كل منتج، حيث يتم زيادة قيمة المنتج عند إتمام عملية شراء جديدة باستخدام الأمر:

```
Redis::zincrby('popular_products', 1, $productId);
```

يسمح هذا الأسلوب بالحصول على المنتجات الأكثر طلباً بسرعة كبيرة دون الحاجة إلى تنفيذ عمليات تجميع (Aggregation) مكلفة على قاعدة البيانات.

كما تم استخدام Laravel Cache لتخزين قائمة المنتجات الأكثر طلباً لمدة ساعة كاملة، وذلك من خلال:

```
Cache::remember(
    'top_products_list',
    3600,
    function () {
        ...
    }
);
```

عند تحديث عدد الطلبات الخاصة بأي منتج يتم حذف النسخة المخزنة تلقائياً:

```
Cache::forget('top_products_list');
```

مما يضمن المحافظة على تزامن البيانات بين Redis وقاعدة البيانات.

آلية العمل

تمر عملية تخزين المنتجات الأكثر طلباً بالمراحل التالية:

1. إتمام عملية شراء منتج.
2. زيادة قيمة المنتج داخل Redis Sorted Set.
3. حذف النسخة المخزنة مؤقتاً من قائمة المنتجات.
4. عند طلب قائمة المنتجات الأكثر طلباً يتم استرجاعها من Redis.
5. يتم تخزين النتيجة داخل Cache لمدة ساعة.
6. الطلبات اللاحقة تحصل على البيانات مباشرة من Cache دون الرجوع إلى قاعدة البيانات.

لتخزين سلة التسوق Redis استخدام 6.4

تم الاعتماد على Redis لتخزين محتويات سلة التسوق الخاصة بالمستخدمين بدلاً من قاعدة البيانات التقليدية وذلك بهدف تقليل زمن الاستجابة وتحسين أداء النظام عند التعامل مع عدد كبير من المستخدمين المتزامنين.

لكل مستخدم يتم إنشاء مفتاح مستقل داخل Redis بالشكل التالي:

```
cart:user:{user_id}
```

ويتم تخزين عناصر السلة باستخدام بنية Hash بحيث تحتوي على:

- معرف المنتج
- اسم المنتج
- السعر الحالي
- الكمية المطلوبة

كما تم تحديد مدة صلاحية تلقائية للسلة تبلغ سبعة أيام:

```
Redis::expire($key, 86400 * 7);
```

مما يمنع تراكم البيانات غير المستخدمة داخل Redis.

توفر الخدمة العمليات الأساسية التالية:

- إضافة منتج إلى السلة
- إزالة منتج من السلة
- تعديل الكمية
- عرض محتويات السلة
- تفريغ السلة
- تنفيذ عملية الشراء (Checkout)

اختبار النظام

تم اختبار طبقة التخزين المؤقت باستخدام Postman وكانت النتائج كما يلي:

إضافة منتج إلى السلة

Endpoint

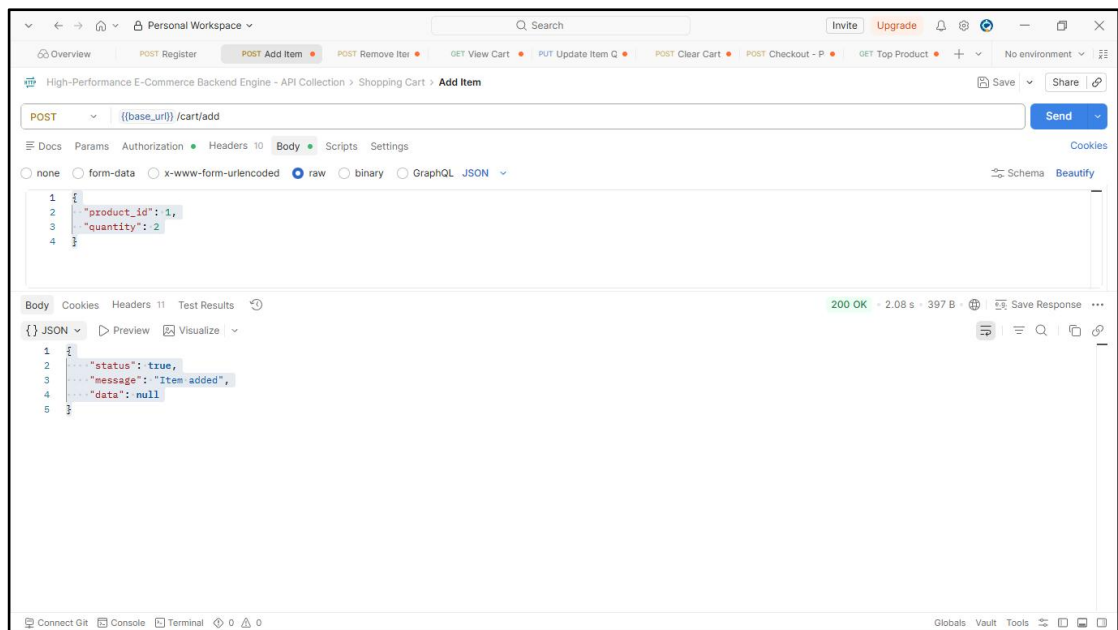
```
POST /cart/add
```

Body

```
{  
  "product_id": 1,  
  "quantity": 2  
}
```

النتيجة

- Status: 200 OK
- تم إضافة المنتج بنجاح إلى Redis Cart
- زمن الاستجابة: 4.13 ثانية



إضافة منتج إلى سلة التسوق المخزنة في Redis

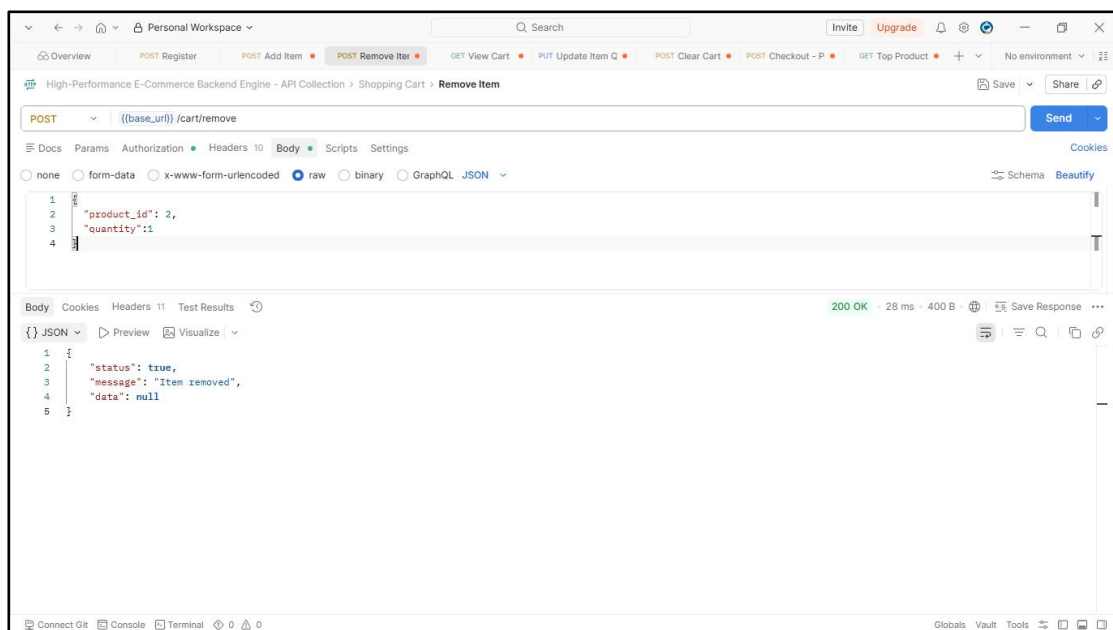
إزالة منتج من السلة

Endpoint

POST /cart/remove

النتيجة

- Status: 200 OK
- تم حذف المنتج من Redis Cart



إزالة منتج من سلة التسوق

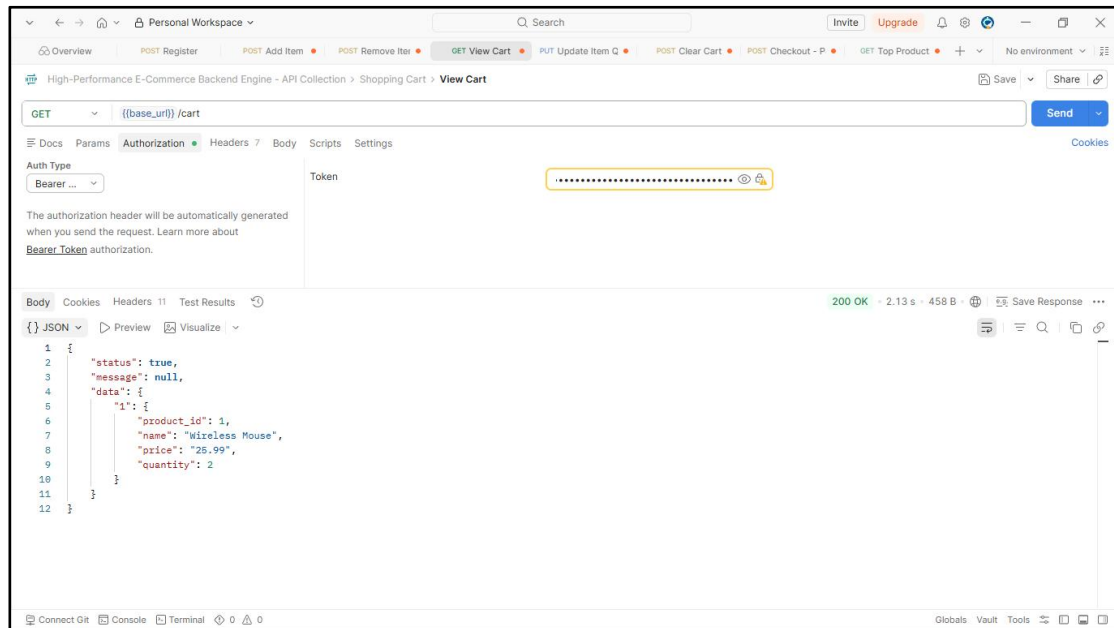
عرض محتويات السلة

Endpoint

GET /cart

النتيجة

- Status: 200 OK
- تم استرجاع بيانات السلة من Redis مباشرة
- احتوت السلة على المنتج بكمية 2 Wireless Mouse



استرجاع بيانات السلة من Redis

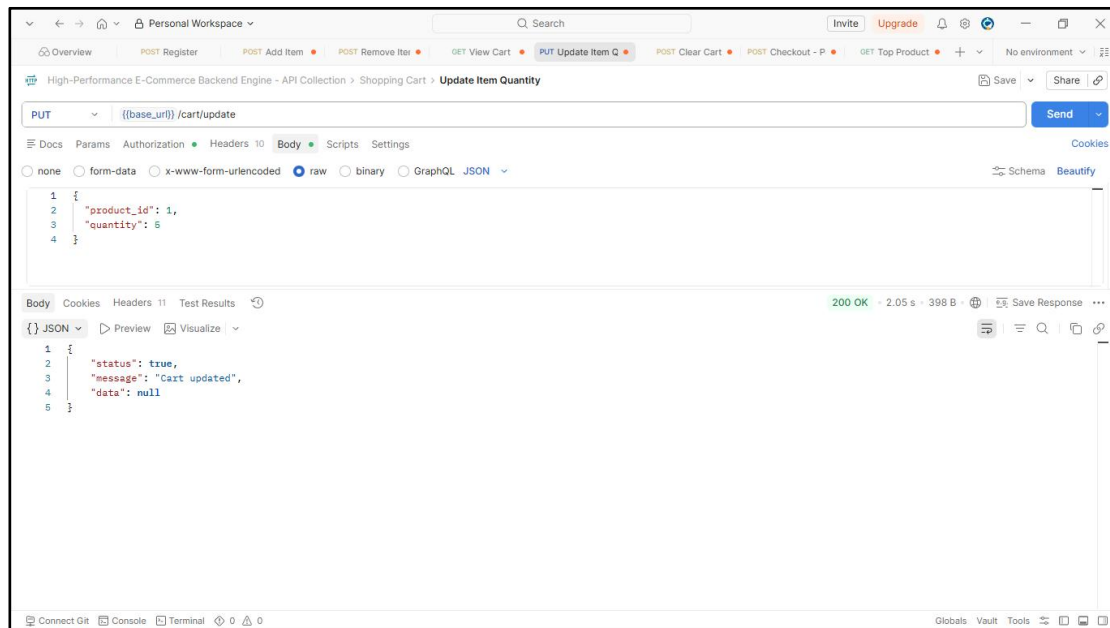
تحديث كمية منتج

Endpoint

PUT /cart/update

النتيجة

- Status: 200 OK
- تم تعديل الكمية بنجاح داخل Redis



تحديث بيانات عنصر داخل السلة

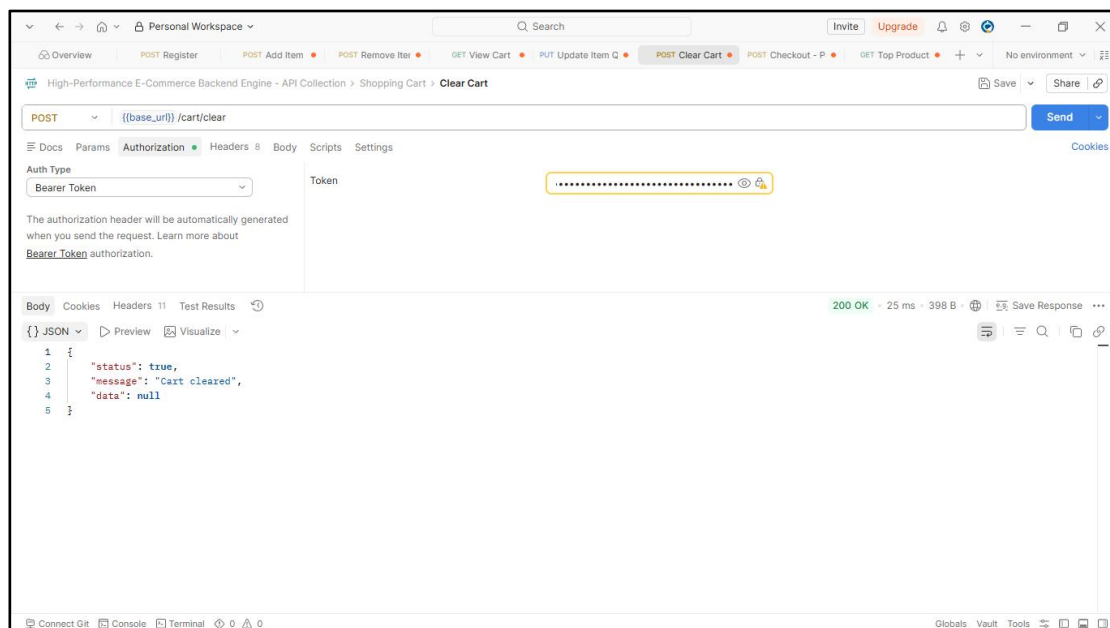
تفريغ السلة

Endpoint

POST /cart/clear

النتيجة

- Status: 200 OK
- تم حذف جميع عناصر السلة من Redis.



حذف جميع عناصر سلة التسوق

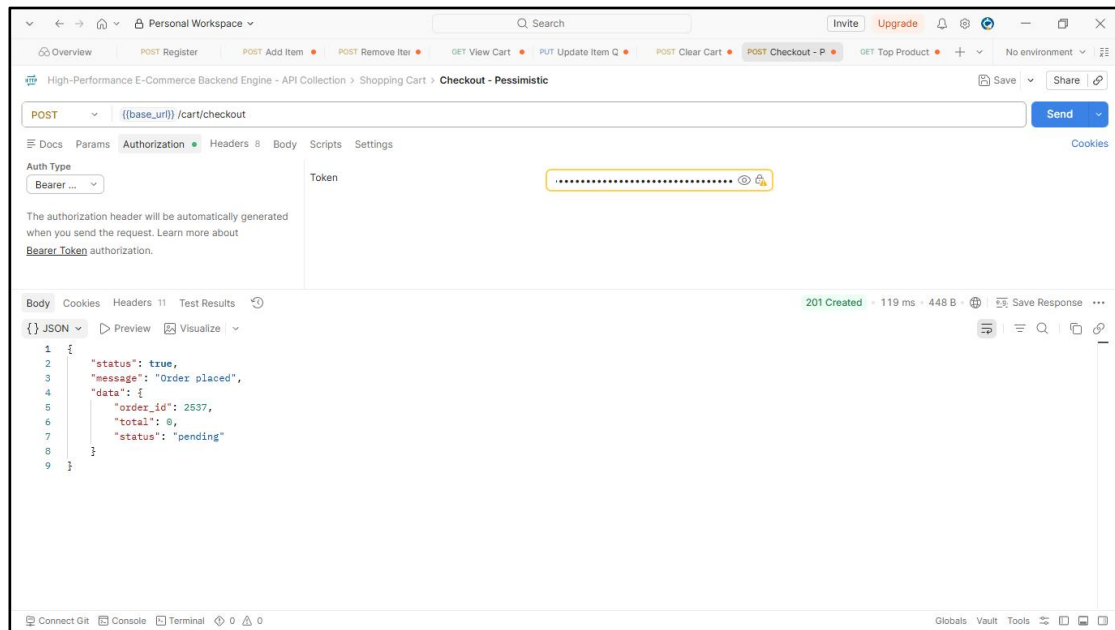
تنفيذ عملية الشراء

Endpoint

POST /cart/checkout

النتيجة

- Status: 201 Created
- تم إنشاء الطلب بنجاح
- رقم الطلب: 2537
- الحالة الابتدائية: Pending.



إنشاء طلب جديد انطلاقاً من Redis Cart

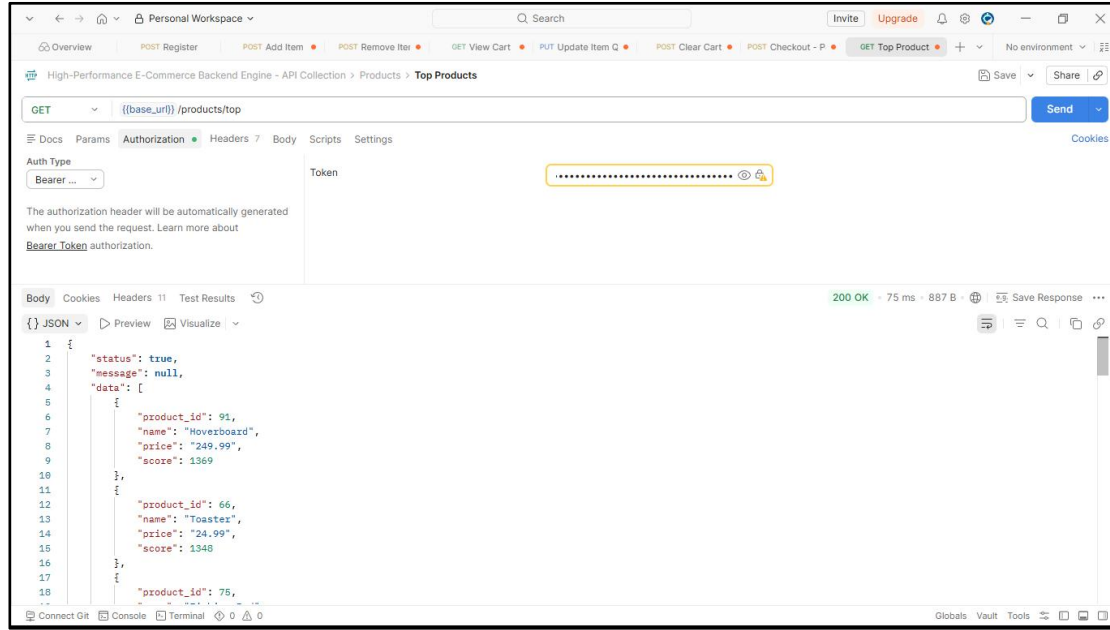
استرجاع المنتجات الأكثر طلباً

Endpoint

GET /products/top

النتيجة

- Status: 200 OK
- تم استرجاع قائمة المنتجات الأكثر طلباً من Redis Cache
- احتوت النتائج على المنتجات الأعلى مبيعاً مع قيمة Score الخاصة بكل منتج



استرجاع المنتجات الأكثر طلباً من Redis Cache

تحليل النتائج

أظهرت النتائج نجاح دمج Redis كطبقة تخزين مؤقت موزعة داخل النظام، حيث تم تحقيق ما يلي:

- تقليل عدد الاستعلامات المباشرة على قاعدة البيانات
- تسريع عمليات إدارة سلة التسوق
- تسريع استرجاع المنتجات الأكثر طلباً
- تخفيف الحمل على خوادم قاعدة البيانات
- تحسين قابلية التوسع عند زيادة عدد المستخدمين

كما ساهم استخدام Redis Cache مع Redis Sorted Set في توفير آلية فعالة لتتبع المنتجات الأكثر طلباً مع المحافظة على زمن استجابة منخفض.

الاستنتاج

تم تطبيق استراتيجية Distributed Caching باستخدام Redis بنجاح لتخزين بيانات سلة التسوق والمنتجات الأكثر طلباً. وقد ساعد ذلك على تحسين أداء النظام وتقليل الضغط على قاعدة البيانات، مما يجعل النظام أكثر قدرة على التعامل مع الأحمال المرتفعة والطلبات المتزامنة مع الحفاظ على سرعة الاستجابة.

التحكم في التزامن (Concurrency Control)

مقدمة

تُعد مشكلة التزامن (Concurrency) من أكثر المشكلات أهمية في أنظمة التجارة الإلكترونية، حيث يمكن أن يحاول عدة مستخدمين شراء المنتج نفسه في الوقت ذاته، مما قد يؤدي إلى بيع كمية أكبر من المخزون الفعلي (Overselling).

لمنع هذه المشكلة تم تطبيق استراتيجيتين مختلفتين للتحكم بالتزامن:

- القفل التشاؤمي (Pessimistic Locking)
- القفل المتفائل (Optimistic Locking)

تم اختبار كلا الأسلوبين ضمن عمليات إنشاء الطلبات وتحديث كميات المخزون الحساسة.

القفل التشاؤمي (Pessimistic Locking)

يعتمد القفل التشاؤمي على افتراض احتمال حدوث تعارض بين العمليات المتزامنة، لذلك يتم حجز السجل داخل قاعدة البيانات أثناء عملية التعديل.

تم تطبيق ذلك باستخدام:

```
$product = $this->productRepo->findAndLockForUpdate($item['product_id']);
```

ويؤدي هذا الأسلوب إلى:

- منع أي عملية أخرى من تعديل المنتج نفسه.
- ضمان سلامة بيانات المخزون.
- منع حدوث Overselling.
- تنفيذ العمليات بشكل متسلسل على نفس السجل.

كما تم تنفيذ العملية داخل Transaction لضمان الذرية (Atomicity):

```
DB::transaction(function () {  
    ...  
}, 5);
```

القفل المتفائل (Optimistic Locking)

يعتمد القفل المتفائل على وجود حقل Version داخل جدول المنتجات.

عند قراءة المنتج يتم حفظ قيمة Version الحالية، ثم يتم تحديث السجل فقط إذا بقيت القيمة كما هي.

```
$currentVersion = $product->version;
```

ثم يتم تنفيذ التحديث بالشكل التالي:

```
$affected = Product::where('id', $product->id)  
->where('version', $currentVersion)  
->update([  
    'stock' => $newStock,  
    'version' => $currentVersion + 1,  
]);
```

إذا كانت قيمة:

```
$affected === 0
```

فهذا يعني أن عملية أخرى قامت بتعديل السجل قبل انتهاء العملية الحالية، وبالتالي يتم رفض التحديث وإرجاع خطأ تعارض.

```
throw new RuntimeException('Optimistic lock failure, please retry',409);
```

مزايا هذا الأسلوب:

- لا يحجز السجل داخل قاعدة البيانات
- أداء أعلى عند انخفاض نسبة التعارض
- مناسب للأنظمة ذات القراءة المرتفعة

بنية النظام

يدعم النظام أربع طرق مختلفة لمعالجة الطلبات:

Pessimistic Locking

· Pessimistic Locking (Synchronous - No Limit - Unsafe)

- POST /orders/pessimistic/test/no-limit/unsafe

· Pessimistic Locking (Synchronous - No Limit - Safe)

- POST /orders/pessimistic/test/no-limit/safe

· Pessimistic Locking (Synchronous - With Limit - Unsafe)

- POST /orders/pessimistic/test/with-limit/unsafe

· Pessimistic Locking (Synchronous - With Limit - Safe)

- POST /orders/pessimistic/test/with-limit/safe

· Pessimistic Locking (Queue - With Limit - Unsafe)

- POST /orders/queue/pessimistic/with-limit/unsafe

· Pessimistic Locking (Queue - With Limit - Safe)

- POST /orders/queue/pessimistic/with-limit/safe

· Pessimistic Locking (Queue - No Limit - Unsafe)

- POST /orders/queue/pessimistic/no-limit/unsafe

· Pessimistic Locking (Queue - No Limit - Safe)

- POST /orders/queue/pessimistic/no-limit/safe

Optimistic Locking

· Optimistic Locking (Synchronous - No Limit - Unsafe)

- POST /orders/optimistic/test/no-limit/unsafe

· Optimistic Locking (Synchronous - No Limit - Safe)

- POST /orders/optimistic/test/no-limit/safe

· Optimistic Locking (Synchronous - With Limit - Unsafe)

- POST /orders/optimistic/test/with-limit/unsafe

· Optimistic Locking (Synchronous - With Limit - Safe)

- POST /orders/optimistic/test/with-limit/safe

· Optimistic Locking (Queue - With Limit - Unsafe)

- POST /orders/queue/optimistic/with-limit/unsafe

· Optimistic Locking (Queue - With Limit - Safe)

- POST /orders/queue/optimistic/with-limit/safe

· Optimistic Locking (Queue - No Limit - Unsafe)

- POST /orders/queue/optimistic/no-limit/unsafe

· Optimistic Locking (Queue - No Limit - Safe)

- POST /orders/queue/optimistic/no-limit/safe

· Direct Product Stock Update (Optimistic Locking - Unsafe)

- POST /products/optimistic/decrement/unsafe

· Direct Product Stock Update (Optimistic Locking - Safe)

- POST /products/optimistic/decrement/safe

كما تم دعم نفس الآلية ضمن المعالجة غير المتزامنة باستخدام Queue Workers.

اختبار القفل التшаؤمي

تم تنفيذ الطلب التالي:

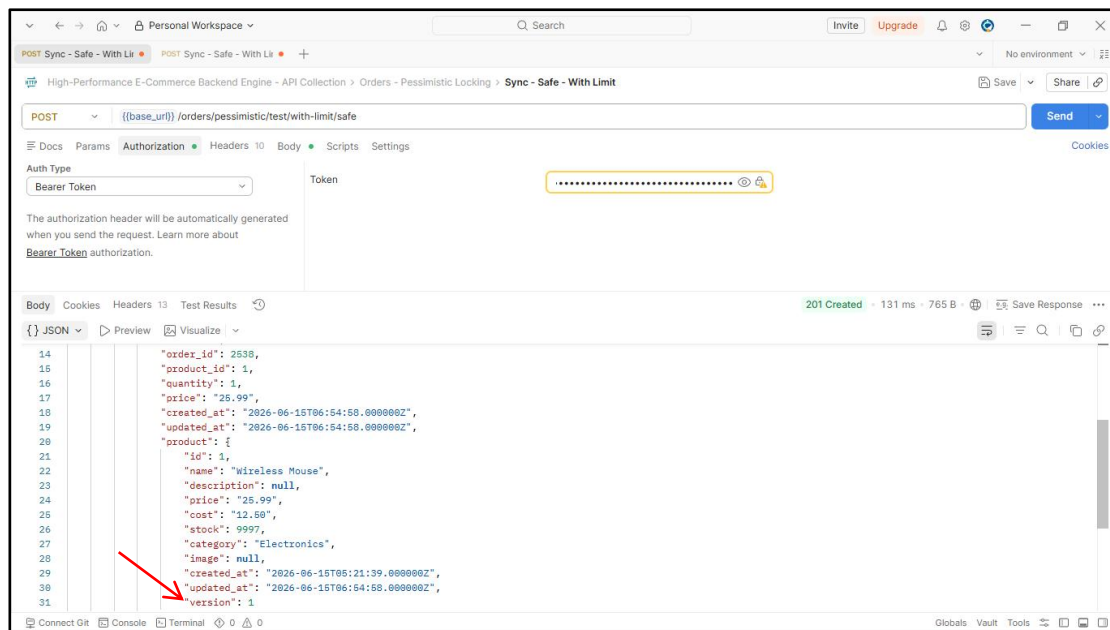
POST /orders/pessimistic/test/with-limit/safe

النتائج

- Status: 201 Created
- تم إنشاء الطلب بنجاح
- Transaction تم تحديث المخزون داخل
- لم يحدث أي تعارض في البيانات
- بقيت قيمة للمنتج 1 = Version

الملاحظات

يعني ذلك أن النظام استخدم Lock على السجل مباشرة ولم يحتاج إلى آلية Versioning



Successful Order Processing Using Pessimistic Locking

اختبار القفل المتفائل

تم تنفيذ الطلب التالي:

POST /orders/optimistic/test/with-limit/safe

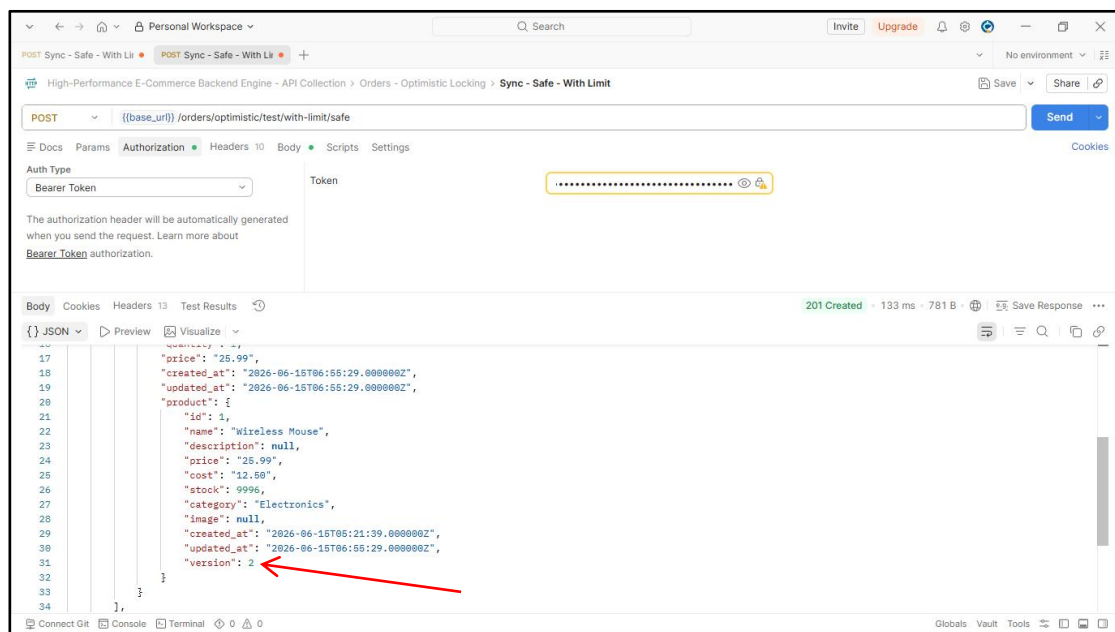
النتائج

- Status: 201 Created
- تم إنشاء الطلب بنجاح.
- تم تعديل المخزون.
- تم زيادة قيمة من 1 إلى 2.

ظهر ذلك في الاستجابة:

"version": 2

مما يؤكد نجاح آلية Optimistic Locking.



Successful Order Processing Using Optimistic Locking

مقارنة بين الأسلوبين

Optimistic Locking	Pessimistic Locking	العنصر
التحقق من Version قبل الحفظ	حجز السجل قبل التعديل	طريقة العمل
ممكنة	منخفضة جداً	احتمالية التعارض
أسرع	أبطأ نسبياً	الأداء
أقل	أعلى	استهلاك الموارد
الأنظمة ذات القراءة المرتفعة	الأنظمة ذات الكتابة المرتفعة	مناسب لـ
نعم	نعم	منع Overselling

تحليل النتائج

أظهرت الاختبارات أن كلا الأسلوبين نجحا في حماية المخزون من التحديثات المتزامنة غير الصحيحة.

القفل التشاؤمي وفر أعلى درجة من الأمان من خلال حجز السجل أثناء التعديل، بينما وفر القفل المتفائل أداءً أفضل من خلال الاعتماد على Version Number واكتشاف التعارض عند الحفظ.

كما أثبتت النتائج أن النظام قادر على التعامل مع الطلبات المتزامنة دون فقدان سلامة البيانات أو حدوث Overselling.

الاستنتاج

تم تطبيق كل من Pessimistic Locking و Optimistic Locking داخل النظام للتحكم بعمليات تحديث المخزون الحساسة.

وقد نجح كلا الأسلوبين في ضمان سلامة البيانات ومنع التعارضات أثناء تنفيذ الطلبات المتزامنة، مما يجعل النظام أكثر موثوقية وقابلية للتوسع في بيئات العمل ذات الأحمال المرتفعة.

سلامة المعاملات (Transaction Integrity / ACID)

مقدمة

تُعد سلامة المعاملات (Transaction Integrity) من أهم المتطلبات في أنظمة التجارة الإلكترونية، حيث يجب ضمان تنفيذ العمليات المترابطة بشكل كامل أو عدم تنفيذ أي جزء منها عند حدوث خطأ.

في هذا المشروع تم الاعتماد على خصائص ACID التي توفرها قاعدة البيانات MySQL مع Laravel Transactions لضمان أن العمليات الحرجة المرتبطة بالطلبات تتم بشكل آمن ومتناسق حتى في حالات الوصول المتزامن.

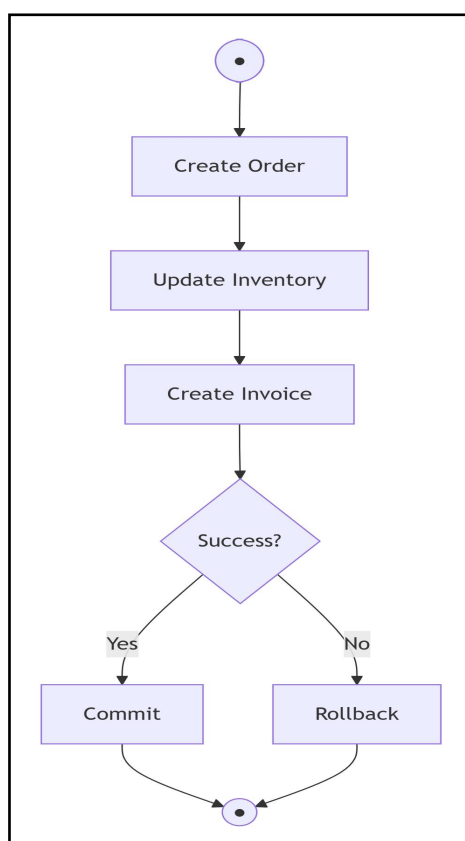
تشمل العمليات الحرجة:

- إنشاء الطلب (Order Creation)
- تحديث المخزون (Inventory Update)
- إنشاء الفاتورة (Invoice Creation)
- تحديث حالة الطلب (Order Status)

معمارية المعاملات

تم تنفيذ جميع العمليات الحرجة ضمن Database Transactions باستخدام Laravel.

شرح المخطط:



**Transaction Integrity
Architecture**

1. **Create Order**: إنشاء الطلب الجديد .
2. **Update Inventory**: خصم الكمية المطلوبة من المخزون
3. **Create Invoice**: إنشاء الفاتورة الخاصة بالطلب .
4. **Transaction Check**:
 1. **Commit**: إذا نجحت جميع العمليات لحفظ التغييرات نهائياً في قاعدة البيانات
 2. **Rollback**: إذا فشلت أي خطوة للتراجع عن التغييرات وإرجاع النظام للحالة السابقة.

هذا المخطط يمثل استخدام Database Transactions

لضمان سلامة البيانات (Data Consistency) بحيث لا يتم إنشاء طلب بدون تحديث المخزون أو إنشاء فاتورة ناقصة.

استخدام Database Transactions

تم استخدام Transaction صريحة داخل خدمة معالجة الطلبات:

وظيفة هذا الجزء:

```

DB::beginTransaction();
try {
    $order = $this->orderService
        ->confirmOrderSafe(
            $order,
            $dto->items
        );
}

```

- بدء معاملة جديدة
- تنفيذ جميع العمليات الحرجة
- اعتماد التعديلات عند النجاح
- التراجع عن جميع التعديلات عند الفشل

```

);

$invoice =
    $this->invoiceService
    ->createFromOrder($order);

DB::commit();
} catch (\Throwable $e) {
    DB::rollBack();

    throw $e;
}

```

تكمال المعاملة مع إدارة المخزون

داخل OrderService تم تنفيذ خصم المخزون وإنشاء عناصر الطلب ضمن Transaction واحدة.

```

DB::transaction(function () use ($order, $items) {
    foreach ($items as $item) {
        $product =
            $this->productRepo
            ->findAndLockForUpdate(
                $item['product_id']
            );

        $product->decrement(
            'stock',
            $item['quantity']
        );
    }
});

```

يضمن هذا الأسلوب:

- منع خصم المخزون بشكل غير صحيح
- منع تضارب الطلبات المتزامنة
- الحفاظ على اتساق البيانات

اختبار التراجع (Rollback Test)

تم إنشاء سيناريو اختبار متعمد للتحقق من سلامة المعاملة.

```
throw new Exception( 'Simulated failure after stock deduction');
```

يتم إطلاق Exception بعد:

- خصم المخزون
- إنشاء الطلب
- إنشاء الفاتورة

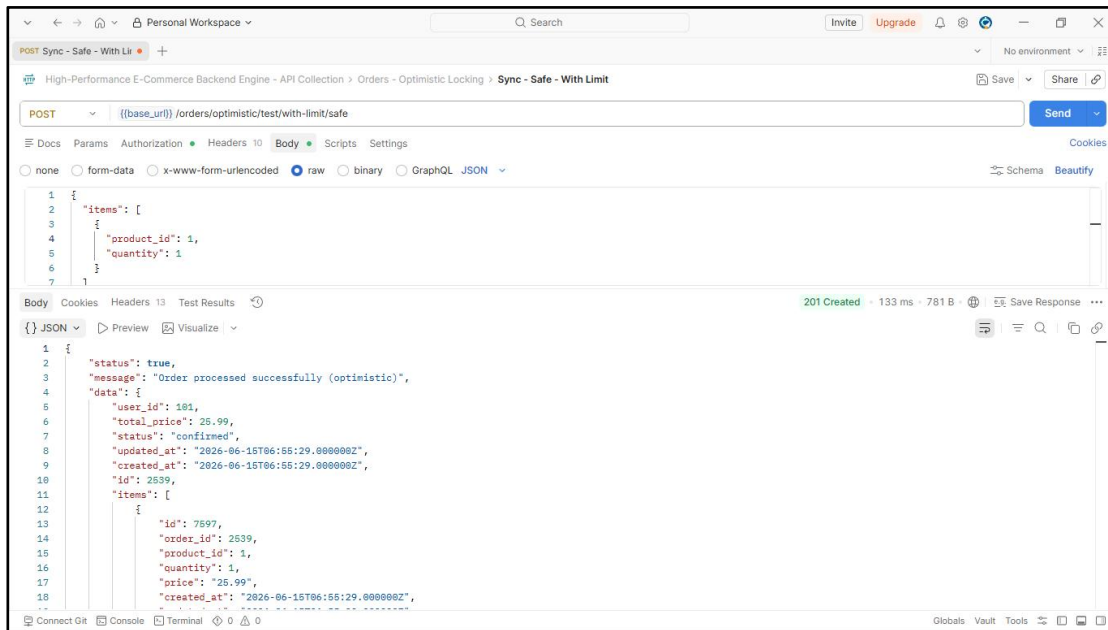
وقبل تنفيذ Commit.

وبالتالي يتم تنفيذ Rollback تلقائياً.

اختبار النظام (Postman Results)

Endpoint

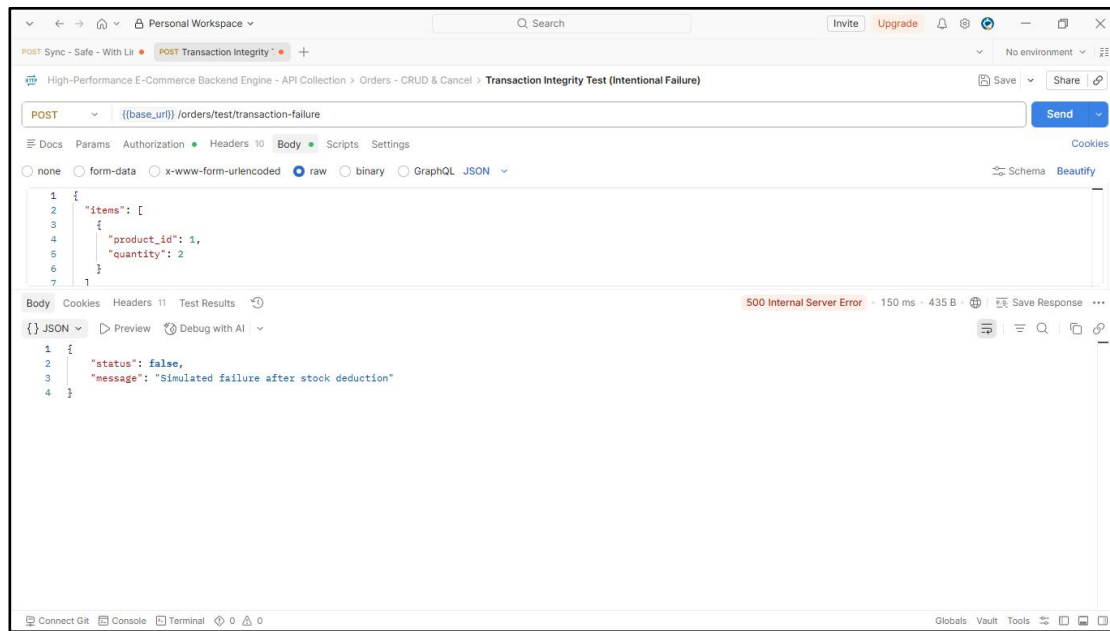
POST /orders/pessimistic/test/with-limit/safe



Successful Transaction Commit

Endpoint

POST /orders/test/transaction-failure



Transaction Rollback Test

اختبار الاستقرار تحت الضغط (Stress Testing)

مقدمة

يهدف اختبار الضغط إلى قياس قدرة النظام على الاستمرار في العمل تحت عدد كبير من الطلبات المتزامنة، والتحقق من عدم حدوث انهيار للنظام أو فقدان للبيانات أثناء التشغيل.

تم استخدام أداة k6 لمحاكاة مستخدمين متزامنين يقومون بالعمليات الأساسية للنظام:

- تسجيل الدخول
- استعراض المنتجات
- إضافة منتجات إلى سلة التسوق
- تنفيذ عملية Checkout
- معالجة الطلبات باستخدام Redis و Queue System

إعدادات الاختبار

تم تنفيذ الاختبار وفق المراحل التالية:

المرحلة	Stage 1	Stage 2	Stage 3	Stage 4
المدة	2 MIN	2 MIN	4 MIN	2 MIN
عدد المستخدمين	60 USERS	120 USERS	120 USERS	نزول تدريجي حتى 0

أقصى عدد مستخدمين متزامنين:

120 Virtual Users (VUs)

وهو أعلى من الحد المطلوب في المشروع:

100 مستخدم متزامن على الأقل.

سيناريو الاختبار

قام كل مستخدم افتراضي بتنفيذ التسلسل التالي

1. تسجيل الدخول باستخدام JWT.
2. جلب قائمة المنتجات.
3. إضافة منتج عشوائي إلى السلة.
4. تنفيذ بنسبة 40% Checkout.
5. الانتظار لفترة قصيرة.
6. إعادة الدورة بشكل متكرر.

نتائج الاختبار

النتائج العامة

المؤشر	القيمة
إجمالي الطلبات	55,296
عدد العمليات المنفذة	23,059
معدل الطلبات	90.13 Request/sec
متوسط زمن الاستجابة	77.15 ms
P95	56.92 ms
أقصى زمن استجابة	8.43 sec
عدد المستخدمين المتزامنين	120

نتائج العمليات و نسبة نجاح الفحوصات

المؤشر	القيمة
--------	--------

53,372	Checks Passed
1904	Checks Failed
96.55%	Success Rate

النجاح	العملية
100%	Get Products
94%	Add To Cart
94%	Checkout

أخطاء الخادم

لم يتم تسجيل أي خطأ من نوع Server Error أثناء الاختبار.

HTTP 5xx Errors = 0%

أي أن النظام لم ينهار.

تحليل النتائج

أظهرت النتائج ما يلي:

- استطاع النظام خدمة 120 مستخدم متزامن
- تم تنفيذ أكثر من 55 ألف طلب خلال الاختبار
- حافظ النظام على زمن استجابة منخفض بمتوسط 77 ms
- لم يتم تسجيل أي أخطاء 5xx
- لم يحدث انهيار للخادم أثناء الاختبار
- استمرت عمليات Redis و Queue Processing بالعمل بشكل طبيعي
- لم يتم ملاحظة فقدان للطلبات أو تعطل في معالجة البيانات

أما نسبة الفشل البالغة 3.44% فهي ليست ناتجة عن انهيار النظام، وإنما تعود إلى حالات عمل متوقعة مثل:

- نفاد المخزون
- محاولة تنفيذ Checkout لسلة فارغة
- تعارضات الأعمال (Business Logic Conflicts)

وهي استجابات صحيحة من النظام وليست أخطاء تقنية.

الاستنتاج

أثبت النظام قدرته على العمل تحت ضغط مرتفع من المستخدمين المتزامنين، حيث نجح في خدمة:

120 مستخدم متزامن

وهو أعلى من الحد الأدنى المطلوب وهو:

100 مستخدم متزامن

كما حافظ على:

- استقرار النظام
- عدم حدوث انهيار

- عدم وجود أخطاء خادم
- زمن استجابة منخفض
- معالجة صحيحة للطلبات المتزامنة

وبذلك يمكن اعتبار متطلب:

Stress Testing (100 Concurrent Users Minimum) محققاً بالكامل ضمن المشروع

```
Administrator: Windows Power... Administrator: Windows Power... Administrator: Windows Power... + v
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> k6 run stress-test-redis.js

Grafana

execution: local
script: stress-test-redis.js
output: -

scenarios: (100.00%) 1 scenario, 120 max VUs, 10m30s max duration (incl. graceful stop):
  * default: Up to 120 looping VUs for 10m0s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

INFO[0613]
+-----+
| Metric | Value | source=console |
+-----+
| Total requests | 55296 | source=console |
| Request rate (req/s) | 90.13 | source=console |
| Average response time (ms) | 77.15 | source=console |
| p95 response time (ms) | 156.93 | source=console |
| p99 response time (ms) | 0.00 | source=console |
+-----+
ERROR[0613] TypeError: Cannot read property 'toString' of undefined or null
running at handleSummary (file:///D:/Studying/Branching Programming/Project/HighPerformanceECommerceBackendEngine/tests/Testers/stress-test-redis.js:165:51(306)) h
int="script exception"

THRESHOLDS

http_req_duration
✓ 'p(95)<3000' p(95)=156.92ms
http_req_failed(status:5xx)
✓ 'rate<0.03' rate=0.00%

TOTAL RESULTS

checks_total ..... 55276 90.095344/s
checks_succeeded ... 96.55% 53372 out of 55276
checks_failed ..... 3.44% 1904 out of 55276

✓ Products fetched
X Item added
  94% - ✓ 21685 / X 1374
X Checkout done
  94% - ✓ 8628 / X 530

CUSTOM
add_ok ..... 21685 35.34477/s
checkout_ok ..... 8628 14.862922/s
products_ok ..... 23059 37.584278/s

HTTP
http_req_duration ..... avg=77.15ms min=7.92ms med=59.36ms max=8.43s p(90)=125.69ms p(95)=156.92ms
{ expected_response:true } ..... avg=77.34ms min=12.99ms med=59.88ms max=8.43s p(90)=126.14ms p(95)=157.35ms
http_req_failed ..... 3.44% 1904 out of 55296
{ status:5xx } ..... 0.00% 0 out of 0
http_reqs ..... 55296 90.127942/s

EXECUTION
iteration_duration ..... avg=2.19s min=1.05s med=2.18s max=7.71s p(90)=2.98s p(95)=3.08s
iterations ..... 23059 37.584278/s
vus ..... 1 min=0 max=120
vus_max ..... 120 min=120 max=120

NETWORK
data_received ..... 531 MB 865 kB/s
data_sent ..... 26 MB 43 kB/s

running (10m13.5s), 000/120 VUs, 23059 complete and 0 interrupted iterations
default { [=====] 000/120 VUs 10m0s
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> |
```

K6 Stress Test Results with 120 Concurrent Users

With Cache And Session : Redis

القياس وتحديد الاختناقات (Benchmarking and Bottleneck Analysis)

مقدمة

يهدف تحليل الأداء (Performance Benchmarking) إلى قياس سرعة استجابة النظام تحت الأحمال المختلفة وتحديد نقاط الاختناق (Bottlenecks) التي تحد من قدرة النظام على معالجة الطلبات بكفاءة.

تم تنفيذ هذه الدراسة على نظام التجارة الإلكترونية المطور باستخدام Laravel Octane و Redis و MySQL بهدف تقييم أداء العمليات الأساسية وتحديد أكثر المكونات تأثيراً على سرعة النظام وقابليته للتوسع.

منهجية الاختبار

تم استخدام أداة:

k6 Load Testing

لتوليد أحمال متزايدة على النظام ومحاكاة مستخدمين حقيقيين يقومون بالعمليات التالية:

- استعراض المنتجات
- إضافة منتجات إلى السلة
- تنفيذ Checkout
- إنشاء الطلبات
- تحديث المخزون

تم تنفيذ الاختبارات باستخدام:

العنصر	القيمة
عدد المستخدمين المتزامنين	120
مدة الاختبار	10 د
البيئة	Docker
خادم التطبيق	Laravel Octane (Swoole)
قاعدة البيانات	MySQL
التخزين المؤقت	Redis

دراسة الاختناق قبل التحسين

الوضع الأولي للنظام

قبل تطبيق التحسينات كانت إعدادات النظام كالتالي:

APP_DEBUG=true

SESSION_DRIVER=file

CACHE_DRIVER=file

كان النظام يعتمد على:

- File Cache
- File Sessions

حيث يتم حفظ البيانات المؤقتة داخل الملفات على القرص الصلب.

المشكلة

عند زيادة عدد المستخدمين المتزامنين أصبحت عمليات:

- قراءة الجلسات
- كتابة الجلسات
- الوصول إلى Cache

تتطلب عمليات I/O مستمرة على القرص.

في بيئة متعددة الخوادم تسبب ذلك في:

- زيادة زمن الاستجابة
- زيادة عدد عمليات القراءة والكتابة
- زيادة استهلاك المعالج
- بطء الوصول للبيانات المشتركة

الاختناق المكتشف

تم تحديد الاختناق الرئيسي على أنه:

File-Based Cache & Session Storage

بدلاً من استخدام تخزين داخل الذاكرة (In-Memory Storage).

نتائج القياس قبل التحسين

تم تنفيذ اختبار ضغط وكانت النتائج:

القيمة	العنصر
20,105	Total Requests
32.68 req/s	Request Rate
136 ms	Average Response Time
67 ms	P95 Response Time
9.35%	Failed Requests

تحليل النتائج

لوحظ أن النظام:

- يعالج حوالي 33 طلب فقط في الثانية.
- يفقد جزءاً من الطلبات تحت الضغط.
- يستهلك وقتاً أكبر للوصول إلى البيانات المؤقتة.

استراتيجية المعالجة

لعلاج الاختناق تم تنفيذ مجموعة من التحسينات.

أولاً: نقل الجلسات إلى Redis

قبل:

SESSION_DRIVER=file

بعد:

SESSION_DRIVER=redis

أصبحت جميع الجلسات محفوظة داخل Redis في الذاكرة.

ثانياً: نقل Cache إلى Redis

قبل:

CACHE_DRIVER=file

بعد:

CACHE_DRIVER=redis

أصبحت عمليات التخزين المؤقت تتم داخل الذاكرة بدلاً من الملفات.

ثالثاً: تعطيل Debug Mode

قبل:

APP_DEBUG=true

بعد:

APP_DEBUG=false

مما خفف الحمل الناتج عن توليد رسائل التصحيح.

رابعاً: استخدام Laravel Octane

تم تشغيل النظام باستخدام:

OCTANE_SERVER=swoole

بدلاً من دورة التشغيل التقليدية في PHP.

أدى ذلك إلى:

- إعادة استخدام الذاكرة.
- تقليل وقت الإقلاع لكل Request.
- زيادة Throughput.

دراسة الأداء بعد التحسين

بعد تطبيق التحسينات أعيد تنفيذ نفس اختبار الضغط.

النتائج

العنصر	القيمة
Total Requests	55,296
Request Rate	90.13 req/s
Average Response Time	77 ms
P95 Response Time	156 ms
Failed Requests	3.44%

تحليل النتائج

أظهرت النتائج:

- زيادة عدد الطلبات المنفذة بأكثر من الضعف
- انخفاض متوسط زمن الاستجابة
- تحسن واضح في استقرار النظام
- انخفاض نسبة الأخطاء

المقارنة الرقمية

المؤشر	قبل التحسين	بعد التحسين
Cache Driver	File	Redis
Session Driver	File	Redis
APP_DEBUG	Enabled	Disabled
Average Response Time	136 ms	77 ms
Request Rate	32.68 req/s	90.13 req/s
Total Requests	20,105	55,296
Failure Rate	9.35%	3.44%

حساب نسبة التحسن

تحسن زمن الاستجابة

$$((136 - 77) / 136) * 100 = 43.3$$

أي:

انخفاض زمن الاستجابة بنسبة 43%

تحسن معدل معالجة الطلبات

$$((90.13 - 32.68) / 32.68) * 100 = 176\%$$

أي:

زيادة القدرة الاستيعابية بنسبة 176%

تحسن معدل الأخطاء

$$((9.35 - 3.44) / 9.35) * 100 = 63\%$$

أي:

انخفاض معدل الفشل بنسبة 63%

أثر التحسين على قابلية التوسع

بعد إزالة الاختناق الناتج عن File Cache و File Sessions أصبح النظام:

- أكثر قدرة على العمل ضمن بيئة متعددة الخوادم
- يدعم مشاركة الجلسات بين جميع Instances
- يقلل الضغط على القرص الصلب
- يزيد من سرعة الوصول للبيانات
- يدعم التوسع الأفقي (Horizontal Scaling)

الاستنتاج

أظهرت دراسة Benchmarking أن الاختناق الرئيسي في النظام كان ناتجاً عن استخدام التخزين المعتمد على الملفات للجلسات والتخزين المؤقت.

بعد الانتقال إلى Redis واستخدام Laravel Octane انخفض متوسط زمن الاستجابة بنسبة تقارب 43% وارتفع معدل معالجة الطلبات بنسبة تقارب 176% مع انخفاض واضح في نسبة الأخطاء تحت الضغط.

وبذلك أصبح النظام أكثر كفاءة واستقراراً وقادراً على التعامل مع الأحمال المرتفعة وعدد كبير من المستخدمين المتزامنين مع المحافظة على سلامة البيانات وسرعة الاستجابة.

```
Administrator: Windows Power... Administrator: Windows Power... Administrator: Windows Power... + v
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> k6 run stress-test-redis.js

Grafana

execution: local
script: stress-test-redis.js
output: -

scenarios: (100.00%) 1 scenario, 120 max VUs, 10m30s max duration (incl. graceful stop):
  * default: Up to 120 looping VUs for 10m0s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

WARN[0493] Request Failed error="Get \"http://localhost:8080/api/products\": request timeout"
WARN[0493] Request Failed error="Get \"http://localhost:8080/api/products\": request timeout"
WARN[0493] Request Failed error="Get \"http://localhost:8080/api/products\": request timeout"
WARN[0493] Request Failed error="Get \"http://localhost:8080/api/products\": request timeout"
INFO[0615]
+-----+-----+-----+-----+
| Metric | Value | source=console |
+-----+-----+-----+-----+
| Total requests | 20105 | source=console |
| Request rate (req/s) | 32.68 | source=console |
| Average response time (ms) | 136.06 | source=console |
| p95 response time (ms) | 67.69 | source=console |
| p99 response time (ms) | 0.00 | source=console |
+-----+-----+-----+-----+
INFO[0615]
ERROR[0615] TypeError: Cannot read property 'toString' of undefined or null
running at handleSummary (file:///D:/Studying/Branching Programming/Project/HighPerformanceECommerceBackendEngine/tests/Testers/stress-test-redis.js:165:51(306)) hint="script exception"

THRESHOLDS

http_req_duration
  ✓ 'p(95)<3000' p(95)=67.69ms

http_req_failed{status:5xx}
  ✓ 'rate<0.03' rate=0.00%

TOTAL RESULTS

checks_total.....: 20085 32.649131/s
checks_succeeded....: 90.63% 18205 out of 20085
checks_failed.....: 9.36% 1880 out of 20085

X Products fetched
  L 93% - ✓ 7828 / X 536
X Item added
  L 88% - ✓ 7372 / X 992
X Checkout done
  L 89% - ✓ 3005 / X 352

CUSTOM
add_ok.....: 7372 11.98354/s
checkout_ok.....: 3005 4.884772/s
products_ok.....: 7828 12.72479/s

HTTP
http_req_duration.....: avg=136.05ms min=0s med=26.26ms max=4m36s p(90)=53.71ms p(95)=67.69ms
{ expected_response:true }...: avg=43.33ms min=14.62ms med=27.35ms max=8.62s p(90)=54.11ms p(95)=66.35ms
http_req_failed.....: 9.35% 1880 out of 20105
{ status:5xx }.....: 0.00% 0 out of 0
http_reqs.....: 20105 32.681642/s

EXECUTION
iteration_duration.....: avg=5.48s min=1s med=2.09s max=4m41s p(90)=2.95s p(95)=3.05s
iterations.....: 8364 13.596083/s

vus.....: 1 min=0 max=119
vus_max.....: 120 min=120 max=120

NETWORK
data_received.....: 181 MB 294 kB/s
data_sent.....: 9.5 MB 16 kB/s

running (10m15.2s), 000/120 VUs, 8364 complete and 0 interrupted iterations
default ✓ [=====] 000/120 VUs 10m0s
PS D:\Studying\Branching Programming\Project\HighPerformanceECommerceBackendEngine\tests\Testers> |
```

K6 Stress Test Results with 120 Concurrent Users

With Cache And Session : File

التوثيق المعماري (Architecture Documentation)

مقدمة

يهدف هذا القسم إلى توثيق التصميم البرمجي (Software Architecture) المستخدم في بناء نظام مراقبة الأداء داخل المشروع، مع التركيز على تطبيق مفهوم البرمجة الموجهة للجوانب

(Aspect-Oriented Programming - AOP) لمراقبة أداء النظام بشكل غير تدخلي

(Non-Intrusive Monitoring).

تم تصميم النظام بحيث يتم فصل منطق مراقبة الأداء عن منطق الأعمال (Business Logic) لضمان:

- عدم تداخل المراقبة مع الوظائف الأساسية للنظام
- سهولة التوسعة والصيانة
- توحيد أسلوب تسجيل الأداء عبر جميع الطلبات
- إمكانية تحليل الأداء بشكل مركزي

المعمارية العامة للنظام

يعتمد النظام على معمارية طبقية (Layered Architecture) مع إضافة طبقة مراقبة عرضية

(Cross-Cutting Concern Layer)

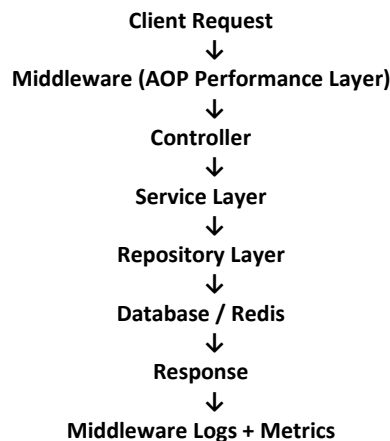
الطبقات الأساسية:

- Presentation Layer (Controllers)
- Service Layer (Business Logic)
- Repository Layer (Data Access)
- Database Layer (MySQL)
- Cache Layer (Redis)

(AOP Layer) الطبقة الإضافية:

- Performance Middleware Layer

الشكل المنطقي للتدفق :



مفهوم AOP في المشروع

تم تطبيق مفهوم البرمجة الموجهة للجوانب (AOP) من خلال Middleware يسمى:

PerformanceMiddleware

الهدف منه:

فصل منطق مراقبة الأداء عن منطق التطبيق الأساسي مثل:

- الحسابات التجارية (Business Logic)
- إدارة الطلبات
- إدارة المنتجات

الوظائف التي يغطيها الـ Middleware

يقوم الـ Middleware بتنفيذ مجموعة من الوظائف العرضية (Cross-Cutting Concerns):

قياس زمن الاستجابة (Response Time)

```
$start = microtime(true);
```

ثم يتم حساب الزمن بعد انتهاء الطلب:

```
$duration = (microtime(true) - $start) * 1000;
```

مراقبة استهلاك الذاكرة

- قياس الذاكرة عند بداية الطلب
- قياس الذاكرة بعد انتهاء الطلب
- حساب الفرق الفعلي

تتبع استعلامات قاعدة البيانات (Query Monitoring)

```
DB::listen(function ($query) {})
```

يتم تسجيل:

- عدد الاستعلامات
- زمن كل استعلام
- أبطأ Query في الطلب

تتبع الطلبات (Request Tracing)

يتم إنشاء معرف فريد لكل طلب:

```
$traceId = Str::uuid()->toString();
```

ويتم تمريره عبر:

- Headers
- Logs
- Response

تسجيل Logs مركزة (Centralized Logging)

يتم تسجيل:

- بداية الطلب (REQUEST_START)
- نهاية الطلب (REQUEST_END)

ويشمل ذلك:

- URL
- Route Name
- User ID
- IP Address
- User Agent
- Response Time
- Memory Usage
- Query Count

تطبيق AOP عملياً

بدلاً من إضافة كود المراقبة داخل كل Controller أو Service:

الطريقة التقليدية :

- تكرار الكود في كل endpoint
- صعوبة الصيانة
- تداخل بين المنطق التجاري والمراقبة

باستخدام AOP Middleware:

تم فصل المراقبة بالكامل داخل طبقة واحدة:

```
class PerformanceMiddleware
```

وهذا يحقق:

- Separation of Concerns
- Clean Architecture
- Centralized Monitoring

تحسينات الأداء الناتجة

بعد تطبيق Middleware:

تم تحقيق:

- رؤية كاملة لأداء كل Request
- تحديد أبطأ Query تلقائياً
- قياس استهلاك الذاكرة بدقة
- تتبع الأخطاء عبر Trace ID
- تحليل الاختناقات بسهولة

آلية تتبع الطلبات (Tracing System)

كل طلب يحصل على: X-Trace-Id: UUID ويتم استخدامه في:

- Logs
- Debugging
- Performance Analysis
- Distributed tracing (future-ready)

مخرجات النظام (Outputs)

:Response Headers

- X-Trace-Id
- X-Response-Time-ms
- X-Memory-Used-kb

أهمية التصميم

هذا التصميم يوفر:

1. Observability

رؤية كاملة للنظام أثناء التشغيل

2. Scalability

قابلية التوسع وتحليل الأداء في بيئات متعددة الخوادم

3. Maintainability

فصل كامل بين:

- Business Logic
- Monitoring Logic

4. Debugging Efficiency

تقليل وقت اكتشاف الأخطاء من خلال Trace ID

الخلاصة

تم تطبيق مفهوم AOP بنجاح من خلال Middleware مخصص لمراقبة الأداء، مما أدى إلى بناء نظام مراقبة غير تدخلي (Non-Intrusive Monitoring System) يوفر رؤية شاملة لأداء النظام دون التأثير على منطق الأعمال. كما ساهم هذا التصميم في تحسين قابلية الصيانة، وتحليل الاختناقات، ودعم عمليات التوسع المستقبلية للنظام. يمكنك إضافة هذه الأقسام في نهاية التقرير أو في فصل الخاتمة

التحديات (Challenges)

واجه المشروع عدداً من التحديات التقنية أثناء التطوير والاختبار، كان أبرزها ضمان سلامة البيانات عند تنفيذ العمليات المتزامنة على المخزون والطلبات، حيث كان من الضروري منع حدوث تعارضات أو فقدان للبيانات عند تنفيذ عدة طلبات في الوقت نفسه. كما تطلبت عملية تحسين الأداء دراسة نقاط الاختناق داخل النظام وتحليل تأثير التخزين المؤقت والجلسات على زمن الاستجابة. بالإضافة إلى ذلك، شكّل اختبار النظام تحت أحمال مرتفعة باستخدام أدوات القياس تحدياً مهماً للتأكد من استقرار التطبيق وقدرته على خدمة عدد كبير من المستخدمين المتزامنين دون التأثير على سلامة البيانات أو توفر الخدمة.

التوصيات (Recommendations)

يوصى مستقبلاً بتوسيع استخدام Redis ليشمل آليات إضافية مثل Distributed Locks وإدارة الطوابير (Queues) عالية الأداء باستخدام Redis Queue بدلاً من قاعدة البيانات. كما يوصى بإضافة أدوات متقدمة لمراقبة الأداء مثل Grafana و Prometheus لمتابعة مؤشرات الأداء بشكل لحظي. ومن المفيد أيضاً تطبيق آليات Auto Scaling عند نشر النظام على بيئات سحابية لدعم الزيادة المفاجئة في عدد المستخدمين. بالإضافة إلى ذلك، يمكن تطوير نظام التنبيهات لمراقبة الاختناقات والأخطاء الحرجة بشكل تلقائي وتحسين تجربة المستخدم واستقرار النظام.

المراجع (References)

1. [Laravel Documentation](#)
2. [Laravel Octane Documentation](#)
3. [Redis Documentation](#)
4. [MySQL Documentation](#)
5. [Grafana k6 Documentation](#)
6. [OWASP Foundation Documentation](#)
7. [Docker Documentation](#)
8. [Nginx Documentation](#)
9. [Martin Fowler – Patterns of Enterprise Application Architecture](#)
10. [Gamma, Helm, Johnson, Vlissides – Design Patterns: Elements of Reusable Object-Oriented Software](#)